

Hochschule Bremerhaven

Fachbereich II
Management und Informationssysteme
Wirtschaftsinformatik B.Sc.

Modul
Vernetzte Systeme

ToDo-Liste

Webanwendung innerhalb eines vernetzten Systems

Vorgelegt von:	Mert Özdemir	MatNr. 41193
	Celal Akköprü	MatNr. 39652
	Mustafa Thamer	MatNr. 39628
	Mohammed Alzubaidy	MatNr. 40085
Vorgelegt am:	10. März 2025	
Dozent:in:	Prof. Dr. Oliver Radfelder und Prof. Dr. Karin Vosseberg	

Inhaltsverzeichnis

1	Einleitung	4
2	Projektorganisation und Teamarbeit	4
3	Erfahrungen und Erkenntnisse aus dem Modul Vernetzte Systeme	5
3.1	Erlernte Kompetenzen und Kenntnisse im Semester	5
3.1.1	Vernetzte Systeme und das Schichtenmodell	5
3.1.2	Webprogrammierung und HTTP	6
3.1.3	Linux-Umgebung und Posix-Standards	6
3.1.4	Datenbanken und Datenverwaltung	6
3.1.5	Docker und Containerisierung	7
4	Technologische Grundlagen	7
4.1	Docker: Grundlagen und Anwendung	7
4.2	Das ISO/OSI-Modell	8
5	Projektidee und Vorhaben	12
5.1	Die Verzeichnisstruktur	14
6	Systemarchitektur	18
6.1	Apache	19
6.2	HAProxy	22
6.3	MariaDB (Datenbank)	26
6.3.1	Datenbank-Erstellung und Benutzerverwaltung	26
6.3.2	Tabellenstruktur für ToDo-Einträge	27
6.3.3	Benutzerverwaltung und Authentifizierung	27
6.4	Redis	28
6.4.1	Logout-Prozess	31
7	Implementierung und Ergebnisse	32
7.1	MariaDB Stresstest	33
7.2	Redis Benchmark	34
7.3	HAProxy Session-Stickiness	35
7.4	HAProxy Load-Balancing Test	36
7.4.1	Erweiterter Load-Balancing Test unter erhöhter Last	37
7.5	Umfassender Netzwerk- und Performance-Test	37
7.6	TCP Dump – Netzwerk- und Performance-Test	38
7.6.1	Erweiterter Netzwerk- und Performance-Test	39
7.7	k6 Lasttest – Performance-Analyse von HAProxy und Apache	40
7.7.1	k6 Login-Lasttest	41

7.7.2	k6 Add-ToDo-Liste	42
7.8	Diagramm – Gnuplot: CPU- und RAM-Nutzungsanalyse	43
8	Fazit und Ausblick	45
	Literaturverzeichnis	46
	Abbildungsverzeichnis	46
	Tabellenverzeichnis	47
	Listingverzeichnis	48
	Anhang	49
I	Die Poster	50
II	Beamer	51
III	Quickbuild	54
	Selbstständigkeitserklärung	55

1 Einleitung

Cloud-Dienste, Datenbanken oder das Internet – all diese Technologien basieren auf der Kommunikation zwischen verschiedenen Systemen. Die Interaktion erfolgt durch eine Client-Host-Architektur, die eine schnelle, stabile und effiziente Datenübertragung sowie die reibungslose Verarbeitung von Anfragen ermöglicht. In der modernen IT-Welt entfernen sich Systeme zunehmend von klassischen Serverräumen und entwickeln sich hin zu vernetzten Server- und Netzwerkarchitekturen.

Die Bedeutung von verteilten und vernetzten Systemen nimmt stetig zu, da sie auf Netzwerken basieren, die die Grundlage der heutigen IT-Infrastruktur bilden. Der Begriff verteilte Systeme bezieht sich in diesem Zusammenhang auf eine Sammlung unabhängiger Einheiten, die miteinander kommunizieren, um ein größeres und effizienteres Gesamtsystem zu bilden. Dies lässt sich mit einer Firma mit verschiedenen Abteilungen vergleichen: Jede Abteilung arbeitet eigenständig, aber durch den kontinuierlichen Austausch von Informationen entsteht eine funktionierende Gesamtstruktur.

Diese Konzepte bilden auch die Grundlage des Moduls "Vernetzte Systeme"(VNS), in dem wir die Möglichkeit hatten, uns intensiv mit der Gestaltung, Implementierung und Verwaltung verteilter Systeme auseinanderzusetzen. Ein zentraler Bestandteil dieser Veranstaltung ist der Einsatz von Docker-Umgebungen, die es ermöglichen, komplexe Anwendungen effizient in Containern zu organisieren und zu betreiben.

Da Teamarbeit eine essenzielle Rolle in der modernen IT-Welt spielt, haben wir uns im Rahmen des VNS-Projekts zu einem Team zusammengeschlossen, um die erlernten Konzepte und Techniken in einem praxisnahen Projekt anzuwenden. Durch diesen praktischen Ansatz möchten wir nicht nur innovative Lösungen entwickeln und die Herausforderungen verteilter Systeme bewältigen, sondern auch unsere Problemlösungsfähigkeiten und Zusammenarbeit im Team weiter verbessern.

2 Projektorganisation und Teamarbeit

Für den Erfolg eines Projekts sind eine gute Organisation und eine effektive Zusammenarbeit im Team unerlässlich. Wir haben viele verschiedene Ideen gehabt, worüber unser Projekt handeln soll. Wir entschieden uns für eine simple ToDo-Liste, da wir uns hauptsächlich auf die verschiedenen Docker-Container fokussieren wollten, damit wir unser gelerntes über vernetzte Systeme und die Relevanz von Docker-Containern selbst und einer Virtual Machine auf die Probe zu stellen. Anschließend haben wir ein Git-Repository erstellt, um eine Basis für unsere Zusammenarbeit zu schaffen. Jedes Teammitglied hatte bestimmte Verantwortungen und Git ermöglichte es uns mehrere Baustellen parallel zu bearbeiten, um das Projekt effizient und zügig

umzusetzen. Wir haben uns für eine flexible Arbeitsmethode entschieden und uns immer gegenseitig ausgeholfen, wodurch jeder an jedem Container und der Webanwendung mitgearbeitet hat. Wir haben aufeinander aufbauend gearbeitet und uns gegenseitig ergänzt, sodass jeder seine eigenen Ideen und Stärken einbringen konnte. In regelmäßigen Teamtreffen, welche sowohl Online als auch Präsenz stattfanden, konnten wir gemeinsam die Richtung des Projekts unter Konsens aller Mitglieder steuern und um eine kontinuierliche Kommunikation sowie einen reibungslosen Arbeitsablauf zu gewährleisten. Alle drei Tage haben wir uns über Jitsi getroffen, um den Fortschritt zu besprechen, Ideen auszutauschen und Probleme gemeinsam zu lösen. Diese regelmäßigen und kurzen Meetings halfen uns, fokussiert zu bleiben und Verzögerungen zu vermeiden. Zusätzlich dazu haben wir uns alle zwei Wochen in der Hochschule getroffen, um komplexere Themen zu diskutieren, den Code gemeinsam zu überprüfen und konkrete Pläne für die nächsten Schritte zu erstellen. Abseits der Teammeetings haben wir uns per Chat-Messenger stets über Änderungen und potenzielle Anregungen oder Ideen informiert, da uns eine offene Kommunikation und eine transparente Arbeitsweise enorm wichtig war. Durch diese klare Struktur und regelmäßigen Meetings konnten wir das Projekt effizient verwalten und sicherstellen, dass alle Teammitglieder aktiv beteiligt waren. Die Kombination aus flexiblen Online-Meetings und persönlichen Treffen ermöglichte uns eine perfekte Balance zwischen Produktivität und Zusammenarbeit. Außerdem haben wir unseren Fortschritt sukzessiv dokumentiert, was es uns erleichterte, den Überblick über erledigte und noch offene Aufgaben zu behalten. Am Ende half uns dieser strukturierte Ansatz, das Projekt erfolgreich und ohne größere Hindernisse abzuschließen.

3 Erfahrungen und Erkenntnisse aus dem Modul Vernetzte Systeme

3.1 Erlernte Kompetenzen und Kenntnisse im Semester

Das Modul „Vernetzte Systeme“ hat uns in diesem Semester eine Vielzahl von Konzepten, Technologien und praktischen Anwendungen nähergebracht. Der Fokus lag darauf, Systeme zu entwerfen, die über mehrere Rechner oder Prozesse verteilt sind und effizient miteinander kommunizieren können. Dazu haben wir verschiedene Werkzeuge und Technologien kennengelernt, darunter Docker, WebSockets, TCP/IP, das ISO/OSI-Modell, SQL-Datenbanken, HTTP-Protokolle und Webprogrammierung.

3.1.1 Vernetzte Systeme und das Schichtenmodell

Ein zentraler Bestandteil des Moduls war das Verständnis der Grundlagen vernetzter Systeme. Wir haben gelernt, dass moderne Softwarelösungen nicht mehr auf einzelnen Servern

laufen, sondern oft auf verteilten Systemen basieren. Diese Systeme bestehen aus mehreren Komponenten, die miteinander kommunizieren und Daten austauschen müssen.

3.1.2 Webprogrammierung und HTTP

Da moderne Anwendungen oft über das Web kommunizieren, haben wir das HTTP-Protokoll detailliert behandelt. Wir haben uns mit HTTP, Cookies und Sessions beschäftigt und gelernt, wie Webserver und Clients miteinander kommunizieren.

Mit HTML, CSS und JavaScript konnten wir eigene Webanwendungen erstellen, die über AJAX dynamisch Daten nachladen. Dies war besonders wichtig für Anwendungen, die in Echtzeit mit dem Server kommunizieren müssen – ein Bereich, in dem WebSockets eine große Rolle spielen.

Dafür haben wir uns das ISO/OSI-Schichtenmodell sowie den TCP/IP-Stack angesehen. Diese Modelle helfen dabei, Netzwerke strukturiert zu verstehen und Protokolle gezielt einzusetzen. Besonders wichtig war die Transportschicht (TCP/UDP), da sie für die zuverlässige Kommunikation zwischen Servern und Clients sorgt.

3.1.3 Linux-Umgebung und Posix-Standards

Da viele vernetzte Systeme auf Linux-Servern betrieben werden, haben wir uns intensiv mit Linux-Befehlen und der Arbeit in der Bash-Shell beschäftigt. Wir haben uns die Umgebung bereits im vorherigen Semester mehrmals aufgesetzt und gelernt mit den Tools umzugehen, um Daten effizient zu verarbeiten. Somit konnten wir alle mit der gleichen Umgebung arbeiten, unabhängig vom Betriebssystem des privaten Endgeräts. Ein weiteres wichtiges Konzept war Posix (Portable Operating System Interface). Posix stellt sicher, dass Software auf verschiedenen Unix-basierten Systemen lauffähig bleibt. Da Docker-Container ebenfalls auf Posix basieren, erleichtert dies die Portabilität von Anwendungen.

3.1.4 Datenbanken und Datenverwaltung

Ein weiterer großer Themenbereich war der Umgang mit SQL-Datenbanken (MariaDB). Wir haben bereits im vorherigen Semester gelernt, wie man Datenbanken anlegt, Tabellen erstellt, Daten einfügt, aktualisiert und abfragt. Auf dieses Wissen haben wir dieses Semester aufgebaut und unsere Kenntnisse verbessert.

Zusätzlich haben wir untersucht, wie sich Datenbank-Architekturen in vernetzten Systemen optimieren lassen. Hierzu gehörten Konzepte wie Datenbank-Sharding und Caching mit Redis.

3.1.5 Docker und Containerisierung

Ein zentrales Thema des Semesters war die Container-Technologie Docker. Wir haben gelernt, inwiefern Docker uns hilft, Anwendungen und ihre Abhängigkeiten isoliert in Containern bereitzustellen. Dies erleichtert die Skalierung und den Betrieb verteilter Systeme erheblich. In unserem Projekt haben wir Docker intensiv genutzt, um unsere Webanwendung in einer vernetzten Umgebung aufzubauen. Die Grundlage die wir genutzt haben wurde freundlicherweise von unserem Dozenten zur Verfügung gestellt. In diesem Verzeichnis sind die Fundamente, auf denen wir unser eigenes Projekt aufbauen konnten, enthalten. Durch das bereits funktionsfähige Vernetzte System, wie z.B build-all.sh oder die bin/ und context/ Verzeichnisse der einzelnen Docker-Container konnten wir uns nach herzenslust ausprobieren und immer wieder von Neuem beginnen, wenn wir uns in eine Sackgasse gearbeitet haben.

4 Technologische Grundlagen

4.1 Docker: Grundlagen und Anwendung

Docker ist eine Open-Source-Plattform, die zur Containerisierung von Anwendungen dient. Anstatt Software direkt auf einem Betriebssystem zu installieren, verpackt Docker Anwendungen in Container, die alles enthalten, was für ihre Ausführung benötigt wird. Dies macht die Bereitstellung und den Betrieb von Software deutlich einfacher und effizienter. Seit der Veröffentlichung ist Docker der neue Standard geworden für private und kommerzielle Zwecke, die heutzutage nicht mehr wegzudenken sind. Docker basiert auf mehreren zentralen Komponenten, darunter Docker-Images, die als Baupläne für Container dienen und die gesamte Software-Umgebung definieren, sowie Docker-Container, die laufende Instanzen dieser Images sind und die Anwendung tatsächlich ausführen. Zusätzlich ermöglichen Docker-Netzwerke die Kommunikation zwischen verschiedenen Containern, was die Interaktion zwischen Diensten erleichtert, ohne dass diese direkt auf dem Host-System installiert sein müssen.

Einer der größten Vorteile von Docker ist seine Portabilität. Docker-Container können auf jedem System ausgeführt werden, das Docker unterstützt, sei es lokal, in der Cloud oder in einer Testumgebung. Dadurch wird die Anwendungsbereitstellung über verschiedene Umgebungen hinweg vereinfacht. Ein weiterer Vorteil ist die Ressourceneffizienz. Im Gegensatz zu virtuellen Maschinen, die ein vollständiges Betriebssystem für jede Anwendung benötigen, teilen sich Docker-Container das Host-Betriebssystem, was sie deutlich ressourcenschonender macht. Darüber hinaus bietet Docker eine hervorragende Skalierbarkeit, da Anwendungen leicht auf mehrere Container verteilt werden können, um Lasten zu verteilen und die Leistung zu optimieren. Ein weiterer Pluspunkt ist das schnelle Deployment, da Container in Sekunden gestartet und gestoppt werden können, was die Bereitstellungszeiten erheblich verkürzt.

In unserem Projekt haben wir Docker genutzt, um verschiedene Dienste voneinander zu trennen und die Anwendung flexibel zu skalieren. Wir haben beispielsweise den Webserver, bestehend aus Apache und CGI, in einem eigenen Container betrieben, um ihn von anderen Diensten zu isolieren. Die Datenbank, MariaDB, wurde in einem separaten Container ausgeführt, um die Datenverwaltung von der Anwendungslogik zu trennen. Zusätzlich haben wir Redis als Caching-Dienst und zur Sessionverwaltung in einem weiteren Container eingesetzt, um die Leistung unserer Anwendung zu verbessern. Zusätzlich haben wir einen Work-Container für die Lasttests. Durch diese Aufteilung konnten wir die Anwendung effizient skalieren und Lasttests durchführen, ohne uns um Abhängigkeiten oder Konflikte zwischen den Diensten sorgen zu müssen. Docker hat die Entwicklung und das Deployment unserer Anwendung erheblich vereinfacht und beschleunigt.

4.2 Das ISO/OSI-Modell

Das ISO/OSI-Modell ist ein wichtiges Referenzmodell für die Netzwerkkommunikation und besteht aus sieben Schichten, die beschreiben, wie Daten zwischen verschiedenen Systemen übertragen werden. In unserem Projekt, einer ToDo-Liste, die auf Docker-Containern basiert, nutzen wir dieses Modell, um die Kommunikation zwischen den verschiedenen Komponenten wie dem HAProxy-Load-Balancer, dem Apache-Webserver, der MariaDB-Datenbank und dem Redis-Caching-System zu organisieren. Jede dieser Komponenten hat eine spezifische Funktion: Der HAProxy nimmt die eingehenden HTTP-Anfragen entgegen und leitet sie an die Apache-Server weiter, die die Anfragen bearbeiten und die Antworten zurückliefern. MariaDB speichert die Daten der ToDo-Liste, und Redis übernimmt die Sessionverwaltung, um Benutzeranmeldungen effizient zu handhaben. Die Kommunikation zwischen diesen Komponenten erfolgt über verschiedene Schichten des OSI-Modells, was eine klare Trennung und effiziente Verwaltung der Netzwerkprozesse ermöglicht. Auf der Anwendungsschicht laufen beispielsweise die HTTP-Anfragen zwischen dem Client und dem HAProxy sowie die CGI-Skripte für die ToDo-Liste auf den Apache-Servern. Die Transportschicht sorgt mit TCP-Verbindungen durch die Bi-direktionale Kommunikation für eine zuverlässige Verbindung zwischen HAProxy und den Apache-Servern, während die Internetschicht die Datenpakete über IP-Adressen im Docker-Netzwerk routet. In der Sicherungsschicht werden Ethernet-Frames verwendet, um die Daten innerhalb des Netzwerks zuverlässig zu übertragen. Durch diese klare Strukturierung können wir die Kommunikation zwischen den Komponenten effizient verwalten und sicherstellen, dass die Datenübertragung reibungslos funktioniert.

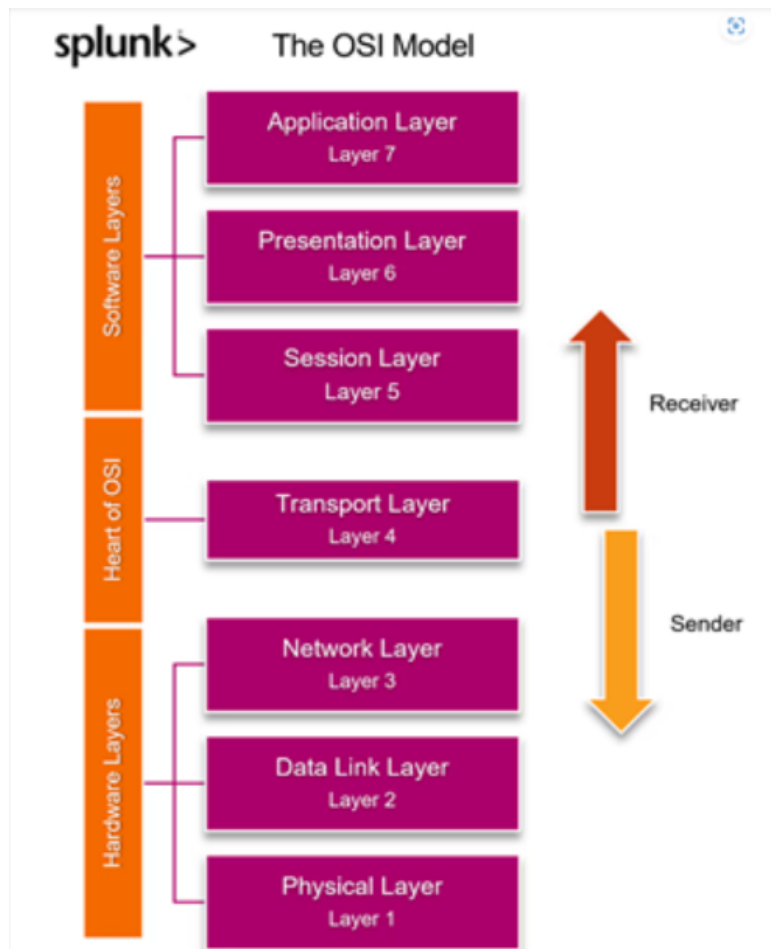


Abbildung 4.1: OSI-Modell

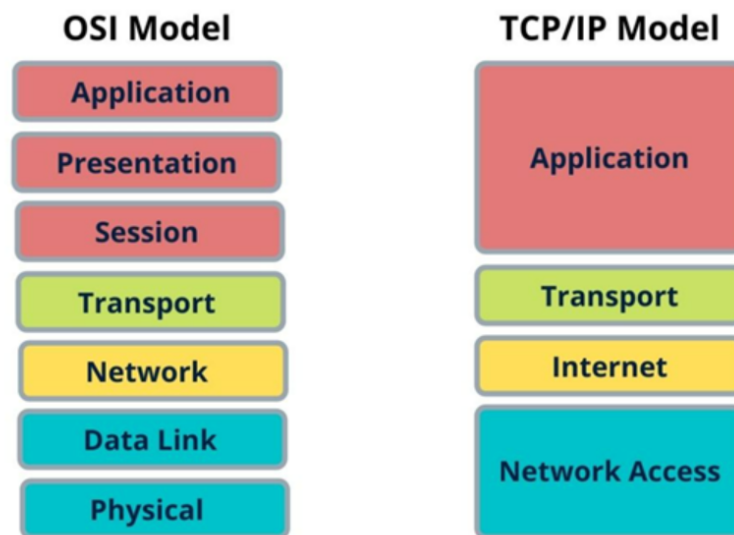


Abbildung 4.2: OSI-Modell und TCP/IP Modell

Die erste Schicht, der Physical Layer, bezieht sich auf die physische Übertragung von Daten. In unserem Projekt läuft die gesamte Infrastruktur auf einem Host-System, das entweder über ein kabelgebundenes Netzwerk (Ethernet) oder eine WLAN-Verbindung mit dem Internet verbunden ist. Innerhalb der Docker-Umgebung existiert jedoch keine physische Verbindung, sondern eine virtuelle Netzwerkarchitektur, die auf der Netzwerkinfrastruktur des Host-Systems aufbaut. Dadurch können die Container miteinander kommunizieren, ohne dass eine direkte physische Verbindung zwischen ihnen besteht.

Die zweite Schicht, der Data Link Layer, sorgt für die fehlerfreie Übertragung von Datenpaketen innerhalb eines lokalen Netzwerks. In unserem Projekt ermöglicht Docker dies durch virtuelle Netzwerkbrücken wie `docker0`, die eine sichere interne Kommunikation zwischen den Containern gewährleisten. Diese Netzwerkbrücken verwalten virtuelle MAC-Adressen und steuern den Datenfluss zwischen den Containern. Da in virtuellen Netzwerken keine echten Kollisionen auftreten, nutzt Docker Switching-Mechanismen zur Datenübertragung.

Die dritte Schicht, der Network Layer, ist für die Adressierung und das Routing von Datenpaketen verantwortlich. In unserem Projekt erhält jeder Docker-Container eine eigene IP-Adresse, die zur Kommunikation mit anderen Containern genutzt wird. Der HAProxy-Load-Balancer verteilt eingehende Anfragen anhand dieser IP-Adressen an verschiedene Backend-Dienste. Die gesamte Kommunikation zwischen Webserver, Datenbank und Caching-System erfolgt innerhalb eines privaten Docker-Netzwerks, das eine sichere und isolierte Umgebung für den Datenaustausch bietet.

Die vierte Schicht, der Transport Layer, stellt eine zuverlässige Ende-zu-Ende-Kommunikation sicher und entscheidet, ob eine Verbindung verbindungsorientiert (z. B. TCP) oder verbindungslos (z. B. UDP) aufgebaut wird. In unserem Projekt nutzen wir ausschließlich TCP (Transmission Control Protocol), da unser System eine stabile und fehlerfreie Datenübertragung benötigt. Der Apache-Webserver empfängt HTTP-Anfragen über TCP und leitet diese an die Datenbank oder das Caching-System weiter. Auch MariaDB nutzt TCP für die Übertragung von SQL-Abfragen, um eine zuverlässige Kommunikation mit der Anwendung sicherzustellen. Redis setzt ebenfalls auf TCP, um schnelle und stabile Anfragen zwischen dem Caching-System und der Anwendung zu ermöglichen. Besonders wichtig in dieser Schicht ist der Three-Way-Handshake (SYN, SYN-ACK, ACK), der für jede neue TCP-Verbindung zwischen Client und Server durchgeführt wird. Dieser Mechanismus sorgt dafür, dass die Datenübertragung zuverlässig ist und keine Pakete verloren gehen.

Die fünfte Schicht, der Session Layer, ist für die Verwaltung und Aufrechterhaltung von Sitzungen zwischen Client und Server verantwortlich. In unserem Projekt spielt Redis eine wichtige Rolle bei der Sitzungsverwaltung, indem es temporäre Login-Informationen speichert. Dadurch bleibt ein Nutzer auch nach mehreren Anfragen authentifiziert, ohne sich erneut anmelden zu müssen. Allerdings übernimmt Redis hier eher eine anwendungsbezogene Funktion, während klassische Sitzungsprotokolle wie Telnet oder SQL-Sitzungen direkte Beispiele für den Session Layer wären.

Die sechste Schicht, der Presentation Layer, sorgt für die Umwandlung, Komprimierung und Verschlüsselung von Daten. Zudem sorgt HTTPS (TLS/SSL) für eine sichere Übertragung, indem es die Daten verschlüsselt, sodass sie nicht von Dritten abgefangen werden können. Die eigentliche Verschlüsselung findet auf der Transportebene durch TLS statt, wird aber oft mit der Präsentationsebene in Verbindung gebracht.

Die siebte Schicht, der Application Layer, ist die Schnittstelle zwischen dem Nutzer und der Anwendung. In unserem ToDo-Listen-Projekt erfolgt der Zugriff über eine Webseite mit HTML, CSS, JavaScript und CGI-Skripten. Der Apache-Webserver verarbeitet die eingehenden Anfragen und sendet die entsprechenden Antworten an den Client. REST-APIs ermöglichen die Kommunikation zwischen der Benutzeroberfläche und der Datenbank, indem sie strukturierte Anfragen und Antworten über HTTP bereitstellen.

5 Projektidee und Vorhaben

Im Rahmen des Moduls Vernetzte Systeme haben wir eine Webanwendung zur Verwaltung von ToDo-Listen entwickelt. Die Anwendung ermöglicht es Nutzern, sich mit einem vordefinierten Benutzerkonto anzumelden und persönliche Aufgaben zu erstellen, zu bearbeiten und zu löschen. Ursprünglich war eine Registrierung vorgesehen, wurde jedoch später aus dem Projekt entfernt, da wir uns nicht lange mit der Webanwendung selbst aufhalten lassen wollten sondern die Hauptziele darin legen, die Docker-Architektur zu verstehen, ein vernetztes System aufzubauen und Lasttests durchzuführen.

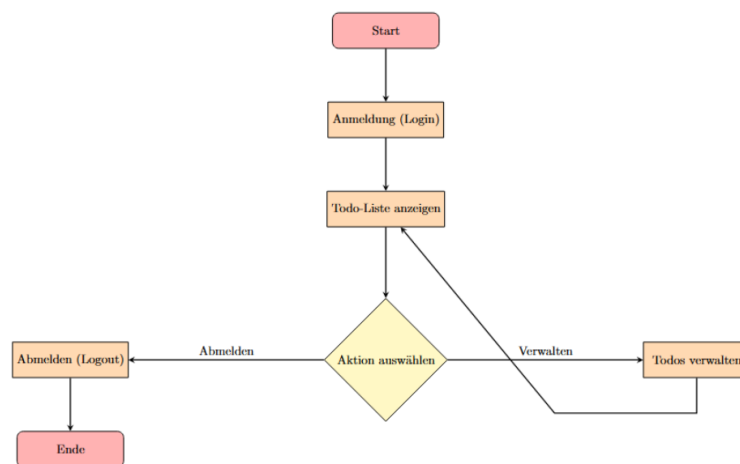


Abbildung 5.1: Die Anwendung

Anfangs lag unser Fokus stark auf der Entwicklung der Webanwendung selbst, da wir die Funktionalität der ToDo-Liste als zentral erachteten. Im Verlauf des Projekts erkannten wir

jedoch, dass die eigentliche Anwendungsentwicklung eher in den Bereich Software Engineering (SWE) fällt, während in diesem Modul die Infrastruktur, Skalierbarkeit und Vernetzung der Systeme im Mittelpunkt stehen. Daraufhin haben wir unsere Herangehensweise angepasst und uns verstärkt mit Containerisierung, Lastverteilung und Session-Management beschäftigt, um die Kernaspekte des Moduls optimal abzudecken.

Ein wesentliches Ziel war die Skalierbarkeit, Ausfallsicherheit und effiziente Lastverteilung der Anwendung. Dies wurde durch die Containerisierung mit Docker, den Einsatz von HAProxy als Load Balancer und die Verwendung von Redis für das Session-Management erreicht. Dank der Sticky Sessions in Redis blieben Nutzer stets mit der gleichen Instanz verbunden, was eine stabile Sitzung gewährleistete. Die Speicherung der Aufgaben erfolgte in einer MariaDB-Datenbank.

Docker spielte eine Schlüsselrolle bei der Bereitstellung und Skalierung der einzelnen Komponenten. Der Webserver wurde mit Apache betrieben, während HAProxy für die Verteilung der Anfragen sorgte. Diese Kombination ermöglichte eine stabile und leistungsfähige Architektur, die sowohl hohe Lasten bewältigen als auch Ausfälle einzelner Instanzen kompensieren konnte. Die Integration von Redis für das Session-Management sorgte für eine schnelle und effiziente Speicherung der Sitzungsdaten, wodurch die Gesamtperformance weiter optimiert wurde.

Durch diesen Technologiestack wurde eine effiziente, skalierbare und robuste ToDo-Listen-Anwendung realisiert. Der Fokus lag darauf, eine belastbare Infrastruktur zu schaffen, die Lastverteilung, Session-Management und Containerisierung optimal integriert und sowohl für den produktiven Einsatz als auch für verschiedene Lasttests genutzt werden konnte.

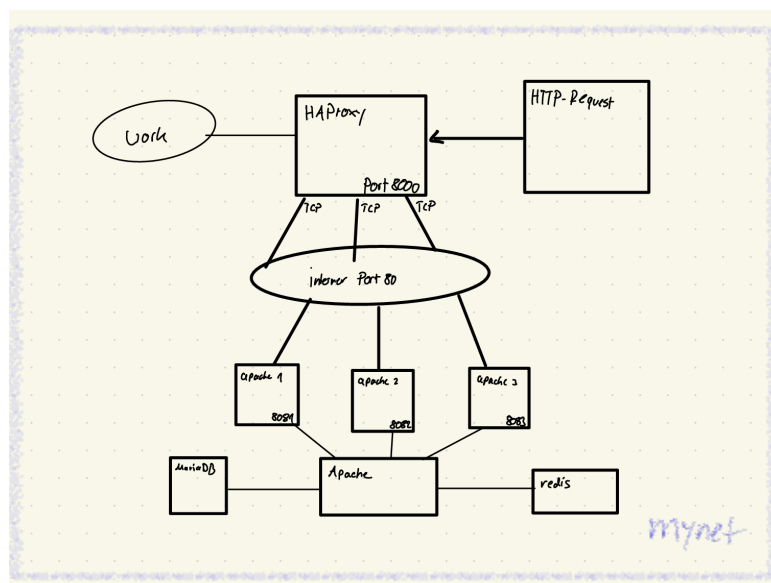


Abbildung 5.2: OSI-Modell und TCP/IP Modell

5.1 Die Verzeichnisstruktur

Unser Docker Verzeichnis hat sich durch die ganzen Komponenten und den dazugehörigen Inhalten ziemlich vergrößert. Im Hauptverzeichnis befindet sich das Skript `build-all.sh`, das den Build-Prozess aller Docker-Container steuert. Darüber hinaus gibt es mehrere Unterverzeichnisse, die jeweils spezifische Komponenten des Systems enthalten.

Das Verzeichnis `docker-apache` beinhaltet die Konfiguration und Skripte für den Apache-Webserver, einschließlich der CGI-Skripte für die Todo-Anwendung. Diese Skripte ermöglichen das Hinzufügen, Bearbeiten und Löschen von Aufgaben sowie das Anmelden und Abmelden von Benutzern. Im Verzeichnis `docker-haproxy` finden sich die Konfigurationsdateien für den HAProxy-Load-Balancer sowie Testskripte und Ergebnisse von Lasttests, die die Leistungsfähigkeit des Systems überprüfen. Für die Datenbankverwaltung ist das Verzeichnis `docker-mariadb` zuständig, das die Docker-Konfiguration für MariaDB sowie Skripte für Stresstests enthält. Ähnlich strukturiert ist das Verzeichnis `docker-redis`, das die Konfiguration für den Redis-Server und Benchmark-Tests bereitstellt. Das Verzeichnis `docker-work` dient als zentrale Anlaufstelle für allgemeine Skripte zur Verwaltung der Docker-Umgebung. Es enthält außerdem Testskripte für Last- und Performance-Tests, die mit Tools wie `k6` durchgeführt werden. Schließlich bietet das Verzeichnis `general` Skripte für die Netzwerkverwaltung, wie das Erstellen und Löschen von Docker-Netzwerken sowie die Bereinigung der gesamten Docker-Umgebung. Des Weiteren haben wir Quality-of-Life Skripte erstellt, wie `start-all.sh` oder `stop-ausgewaehlte-container.sh`, die uns bei beim arbeiten mit den Containern geholfen haben, ohne dass wir zeitliche einbuße hinnehmen mussten.

```
/
├── baue-ausgewaehlte-container.sh
├── build-all.sh
├── docker-apache/
│   ├── bin/
│   │   ├── build.sh
│   │   ├── start.sh
│   │   └── stop.sh
│   ├── context/
│   │   ├── cgi-bin/
│   │   │   └── vns/
│   │   │       ├── test.sh
│   │   │       └── todo/
│   │   │           ├── addTodo.sh
│   │   │           ├── deleteTodo.sh
│   │   │           ├── editTodo.sh
│   │   │           ├── login.sh
│   │   │           ├── logout.sh
│   │   │           ├── my.cnf
│   │   │           ├── styles.css
│   │   │           └── table3.sh
│   │   ├── Dockerfile
│   │   ├── html/
│   │   │   └── index.html
│   │   └── myinit.sh
│   └── docker-haproxy/
│       ├── bin/
│       │   ├── build.sh
│       │   ├── start.sh
│       │   └── stop.sh
│       ├── context/
│       │   ├── Dockerfile
│       │   ├── haproxy.cfg
│       │   ├── lasttest/
│       │   │   ├── haproxy_loadtest.txt
│       │   │   ├── haproxy_load_wrk.sh
│       │   │   ├── haproxy_session_curl.sh
│       │   │   ├── haproxy_session_test.txt
│       │   │   ├── haproxy_sticky_test_1.txt
│       │   │   ├── haproxy_sticky_test_2.txt
│       │   │   └── haproxy_sticky_test_apache1.txt
│       │   └── myinit.sh
```

Abbildung 5.3: Verzeichnisstruktur (Teil 1)

```
/
├── docker-mariadb/
│   ├── bin/
│   │   ├── build.sh
│   │   ├── deploy.sh
│   │   ├── start.sh
│   │   └── stop.sh
│   └── context/
│       ├── Dockerfile
│       ├── lasttest/
│       │   ├── mariadb_stresstest_read.txt
│       │   ├── mariadb_stresstest_write.txt
│       │   └── mariadb_sysbench.sh
│       └── myinit.sh
└── docker-redis/
    ├── bin/
    │   ├── build.sh
    │   ├── start.sh
    │   └── stop.sh
    └── context/
        ├── Dockerfile
        ├── lasttest/
        │   ├── redis_benchmark_results.txt
        │   └── redis_benchmark.sh
        ├── myinit.sh
        └── redis.conf
```

Abbildung 5.4: Verzeichnisstruktur (Teil 2)


```
/
├── docker-work/
│   ├── bin/
│   │   ├── build.sh
│   │   ├── restart.sh
│   │   ├── start.sh
│   │   └── stop.sh
│   ├── context/
│   │   ├── 050_sudo_general
│   │   ├── Dockerfile
│   │   ├── install-k6.sh
│   │   └── lasttest/
│   │       ├── k6_addtodo.sh
│   │       ├── k6_allgemein.sh
│   │       ├── k6_login.sh
│   │       ├── tcpdump_alles.sh
│   │       ├── tcpdump_anylse.sh
│   │       └── tcpdump_logs/
│   │           ├── haproxy_http_test.txt
│   │           ├── haproxy_loadtest.txt
│   │           ├── mariadb_sysbench.txt
│   │           ├── network.pcap
│   │           ├── redis_benchmark.txt
│   │           ├── redis_get.txt
│   │           └── redis_set.txt
│   └── myinit.sh
├── general/
│   ├── create-mynet-network.sh
│   ├── delete-all-containers.sh
│   ├── destroy-mynet-network.sh
│   ├── info.sh
│   └── prune-all.sh
├── start-all.sh
├── starte-ausgewaehlte-container.sh
├── stop-all-containers.sh
└── stop-ausgewaehlte-container.sh
```

Abbildung 5.5: Verzeichnisstruktur (Teil 3)

6 Systemarchitektur

Bei der Systemarchitektur stand für uns im Fokus, welche Docker-Container und wie viele davon benötigt werden, um das gelernte Wissen einzubinden und das Projekt so auszulegen, dass alle Möglichkeiten ausgereizt werden können. Unser Ziel war es, valide und aussagekräftige Testergebnisse zu generieren, die eine tiefgehende Analyse und Auswertung ermöglichen – nicht nur für die Funktionalität, sondern auch für die Skalierbarkeit und Resilienz der Anwendung.

Die Entscheidung, drei identische Apache-Container parallel zu betreiben, fiel aus mehreren Gründen:

HAProxy als Loadbalancer testen: Wir wollten HAProxy nicht nur theoretisch verstehen, sondern praktisch erleben, wie es Last verteilt, auf Ausfälle reagiert und Sessions verwaltet. Herausforderungen wie die korrekte Konfiguration von Sticky Sessions oder die automatische Erkennung inaktiver Backend-Server waren dabei zentral.

Ausfallsicherheit demonstrieren: Ein zweiter und dritter Apache-Container fungieren als redundante Instanz. Fällt einer aus (z. B. durch Neustart oder Fehler), übernehmen die anderen nahtlos – die Webanwendung bleibt für Nutzer uneingeschränkt verfügbar.

Lasttests unter Realbedingungen: Mit Tools wie wrk konnten wir simulieren, wie sich die Anwendung unter hoher Auslastung verhält. Drei Container ermöglichen es, die Grenzen der Skalierbarkeit sichtbar zu machen (z. B. ab wann ein vierter Container nötig wäre).

Durch Redis und MariaDB haben wir zwei weitere zentrale Pfeiler integriert:

MariaDB speichert die Inhalte der Todo-Liste sowie die Login-Daten in strukturierten Tabellen. Die Verbindung zu den Apache-Containern erfolgt über eine gemeinsame my.cnf-Datei im context-Verzeichnis, die zentral Datenbank-Host, Benutzer und Passwörter definiert. Da beide Apache-Container identisch sind, greifen sie problemlos auf dieselbe Konfiguration zu.

Redis verwaltet die Benutzer-Sessions nach dem Login. Hier standen wir vor Herausforderungen, insbesondere bei der persistenten Speicherung der Sessions über Container-Neustarts hinweg und der korrekten Konfiguration des Session-Timeout.

Bevor wir mit dem Testen der Webanwendung begannen, lag unser Fokus auf der korrekten Vernetzung aller Docker-Container. Jeder Container wurde in ein benutzerdefiniertes Netzwerk (mynet) integriert, um die Kommunikation zwischen HAProxy, Apache, MariaDB und Redis zu ermöglichen. Erst nachdem diese Basis stabil lief, haben wir uns an die Feinjustierung der Anwendung gewagt – etwa durch Skripte zur automatisierten Erstellung von Todos oder Lasttests.

Die Vorlagen unseres Dozenten waren dabei ein entscheidender Baustein. Sie lieferten nicht nur das Grundgerüst für die Dockerfiles und Netzwerkkonfiguration, sondern auch Best Practices zur Sicherheit (z. B. Trennung von Frontend- und Backend-Netzwerken) und Ressourcenoptimierung. Dieses Fundament erlaubte es uns, uns schrittweise an komplexere Themen wie

Loadbalancing oder Session-Persistence heranzutasten, ohne von Anfang an in Details verloren zu gehen.

Am Ende steht eine Architektur, die nicht nur funktioniert, sondern auch Raum für Erweiterungen lässt – sei es durch zusätzliche Apache-Container bei steigender Last oder durch die Integration weiterer Dienste wie Websocket oder Ähnliches.

6.1 Apache

Da es keine vorgefertigte Vorlage für einen Apache-Container gab, mussten wir improvisieren und den vorhandenen Tomcat-Container aus dem Docker-Full anpassen, um unsere CGI-Skripte verwenden zu können. Die Umwandlung des Tomcat-Containers in einen Apache-Container war überschaubar – bis auf belegte Ports gab es keine größeren Probleme. Die notwendigen Änderungen beschränkten sich hauptsächlich auf die Dateien `start.sh`, `Dockerfile` und `myinit.sh`.

In der `start.sh` mussten wir lediglich den Namen und den Port anpassen. Da wir zwei identische Apache-Container erstellen wollten, haben wir das Skript so modifiziert, dass es zwei Container erzeugt, anstatt nur einer. Wir haben die Container `apache1` und `apache2` genannt und ihnen die Ports 8081 und 8082 auf dem Host-System zugewiesen.

```
1 #!/bin/bash
2
3 for i in {1..3}; do
4     port=$((8080 + i))
5     docker container create \
6         --name apache$i \
7         --net mynet \
8         --publish $port:80 \
9         --init \
10        --cpus=1 \
11        image-apache
12
13     docker container cp context/myinit.sh apache$i:/usr/bin
14     docker container start apache$i
15
16 done
17
```

Listing 6.1: `start.sh`

Der Schalter `-cpus=1` beschränkt die CPU-Leistung des Containers auf maximal 100%. Die Angabe `:80` hinter dem Port (z. B. `-publish $port:80`) bedeutet, dass der Apache-Server im Container auf dem internen Port 80 läuft. Dies ist der Standard-Port für HTTP-Verkehr. Docker leitet den Verkehr vom externen Port 8081 (oder 8082/8083) auf dem Host-System an den internen Port 80 im Container weiter. Diese Konfiguration war insbesondere für die Integration mit HAProxy wichtig, da HAProxy die beiden Apache-Container über ihren internen Port 80 ansprechen und unter einem gemeinsamen externen Port (in unserem Fall 8000) zusammenführen kann. Um sicherzustellen, dass beide Apache-Container identisch sind und HAProxy die Last korrekt verteilt, führten wir Tests durch. Einen Test konnten wir direkt zu Anfang machen in dem wir einen der Container manuell abgeschaltet haben:

Während die Anwendung unter `localhost:8000` lief, stoppten wir einen der Apache-Container. HAProxy leitete die Anfragen automatisch an den verbleibenden Container weiter, sodass die Anwendung weiterhin verfügbar blieb. Durch diese Konfiguration konnten wir sicherstellen, dass die Last gleichmäßig auf beide Apache-Container verteilt wird und ein Ausfall eines Containers kompensiert werden kann, ohne dass die Verfügbarkeit der Anwendung beeinträchtigt wird.

In der Dockerfile haben wir ebenfalls Anpassungen vorgenommen, um die gewünschten Funktionen der Anwendung umzusetzen. Dazu gehörte unter anderem die Installation von apache-spezifischen Programmen wie `apache2`, `libapache2-mod-fcgid` und `apache2-utils`, die für den Betrieb unserer CGI-Skripte und die Webanwendung notwendig sind. Als Basis-Image nutzten wir deshalb `ubuntu`, um maximale Flexibilität bei der Installation der benötigten Pakete zu haben

Darüber hinaus mussten wir auch Abhängigkeiten (Dependencies) berücksichtigen, die durch die anderen Docker-Container (z. B. MariaDB und Redis) erforderlich sind. Ein Beispiel hierfür ist die Installation von `uuid-runtime`, das für die Generierung von eindeutigen IDs (UUIDs) benötigt wird, die in unserer Anwendung für die Session-Verwaltung oder andere Funktionen genutzt werden könnten.

```
1 FROM ubuntu
2 RUN apt-get -y update && apt-get -y upgrade
3 RUN apt-get -y install apache2 apache2-utils libapache2-mod-fcgid curl vim
   ↪ mariadb-client redis-tools
4 RUN apt-get -y install uuid-runtime
5 COPY cgi-bin/ /usr/lib/cgi-bin/
6 COPY html/ /var/www/html/
7 COPY myinit.sh /usr/bin/myinit.sh
8 RUN chmod +x /usr/bin/myinit.sh
9 RUN chmod +x /usr/lib/cgi-bin/vns/todo/*
```

```
10 RUN a2enmod cgi
11
12 CMD ["/usr/bin/myinit.sh"]
```

Listing 6.2: Dockerfile

Nachdem alle notwendigen Pakete installiert waren, haben wir die Skripte und Dateien der Webanwendung in das Docker-Image kopiert. Dazu gehören:

Die CGI-Skripte, die in das Verzeichnis `/usr/lib/cgi-bin/` kopiert wurden.

Die HTML-Dateien, die in das Verzeichnis `/var/www/html/` kopiert wurden.

Abschließend wurde die `myinit.sh` in das Image integriert und als Hauptprozess des Containers festgelegt.

Die `myinit.sh` ist ein wichtiger Bestandteil unseres Docker-Setups und dient als Init-Skript für den Apache-Webserver. Dieses Skript verwaltet den Start und Stopp des Servers und protokolliert wichtige Ereignisse.

```
1 #!/bin/bash
2 log=/docker.log
3 echo starte >> $log
4
5 trap onexit SIGTERM
6
7 function onexit {
8     #shutdown ...
9     echo shutting down >> $log
10
11     read pid < /run/apache2/apache2.pid
12     apachectl stop
13     while test "$pid" != "" && ps -p $pid >/dev/null; do
14         echo wait for shutdown of pid $pid >> $log
15         sleep 0.01
16     done
17
18     exit
19 }
20 # start services
21 apachectl start
22
23 while true; do
```

```

24     echo "$(date +%FT%T) ping" >> $log
25     read -t 1 </dev/fd/1
26 done

```

Listing 6.3: myinit.sh

Das Skript beginnt mit der Initialisierung einer Log-Datei (/docker.log), in der alle relevanten Ereignisse festgehalten werden. Sobald der Container gestartet wird, schreibt das Skript den Eintrag „starte“ in die Log-Datei, um den Beginn des Prozesses zu dokumentieren.

Ein zentrales Feature der myinit.sh ist die Implementierung eines graceful shutdowns. Mithilfe der trap-Funktion wird das SIGTERM-Signal abgefangen, das vom Docker-Daemon gesendet wird, wenn der Container gestoppt wird. In diesem Fall wird die Funktion onexit aufgerufen, die den Apache-Server ordnungsgemäß herunterfährt. Dazu wird die Prozess-ID (PID) des Apache-Servers aus der Datei /run/apache2/apache2.pid ausgelesen und der Befehl apachectl stop ausgeführt. Das Skript wartet anschließend, bis der Apache-Prozess vollständig beendet ist, und protokolliert den Fortschritt in der Log-Datei.

Nachdem der Apache-Server gestartet wurde, führt das Skript eine Endlosschleife aus, die jede Sekunde einen „ping“-Eintrag mit Zeitstempel in die Log-Datei schreibt. Diese Schleife sorgt dafür, dass das Skript aktiv bleibt und nicht vorzeitig beendet wird. Gleichzeitig ermöglicht sie eine kontinuierliche Überwachung des Containerzustands.

Durch die Integration der myinit.sh in unsere Docker-Infrastruktur wird sichergestellt, dass der Apache-Server zuverlässig startet, korrekt heruntergefahren wird und alle wichtigen Ereignisse protokolliert werden. Dies trägt zur Stabilität und Nachvollziehbarkeit unserer Anwendung bei.

6.2 HAProxy

Der HAProxy-Container bildet das Herzstück der Lastverteilung und sorgt dafür, dass die Anfragen der Nutzer zentral gebündelt und intelligent an die beiden Apache-Container weitergeleitet werden. Da HAProxy selbst als eigenständiger Dienst läuft, mussten wir ein spezielles Docker-Image erstellen, das genau auf diese Anforderungen zugeschnitten ist.

```

1 FROM ubuntu
2 RUN apt-get -y update && apt-get -y upgrade
3
4 RUN apt-get -y install vim tree iproute2 curl w3m ncat less procps
5 RUN apt-get -y install haproxy
6 RUN apt-get -y install sudo wrk

```

```
7 COPY myinit.sh /usr/bin/
8 RUN chmod +x /usr/bin/myinit.sh
9
10 CMD ["/usr/bin/myinit.sh"]
```

Listing 6.4: Dockerfile

In der Dockerfile wurde dazu ein Ubuntu-Basis-Image verwendet, um auch hier wieder maximale Kontrolle über die Installation der benötigten Pakete zu haben. Neben HAProxy selbst wurden auch Tools wie vim, curl und wrk integriert – einerseits für Debugging-Zwecke, andererseits für spätere Lasttests. Das Skript myinit.sh wurde als zentraler Startpunkt festgelegt und im Container unter /usr/bin abgelegt. Durch das Setzen der Ausführungsberechtigungen mit chmod +x wird sichergestellt, dass das Skript beim Containerstart automatisch ausgeführt wird. Die eigentliche Magie passiert jedoch in der myinit.sh. Dieses Skript übernimmt drei Schlüsselaufgaben:

Initialisierung: Beim Start des Containers wird eine Log-Datei (/var/log/docker.log) angelegt, die alle Ereignisse protokolliert – vom Hochfahren des Containers bis hin zu regelmäßigen „Ping“-Signalen, die alle 10 Sekunden geschrieben werden.

Graceful Shutdown: Wird der Container per docker stop beendet, fängt das Skript das SIGTERM-Signal ab. HAProxy wird gezielt gestoppt, und der Container sorgt dafür, dass keine laufenden Verbindungen abrupt unterbrochen werden.

Dienstüberwachung: Über service haproxy status wird der Status des HAProxy-Dienstes in die Log-Datei geschrieben, was später bei der Fehlersuche hilfreich sein kann.

```
1
2 #!/bin/bash
3
4 log=/var/log/docker.log
5 echo "HAProxy-Container gestartet: $(date)" >> "$log"
6
7 function onexit {
8     echo "HAProxy-Container wird heruntergefahren: $(date)" >> "$log"
9     service haproxy stop
10    exit
11 }
12
13 trap onexit SIGTERM
14
15
16 echo "Starte HAProxy..." >> "$log"
```

```
17 service haproxy start
18
19 echo "HAProxy-Status:" >> "$log"
20 service haproxy status >> "$log"
21
22 echo "HAProxy-Container läuft: $(date)" >> "$log"
23 while true; do
24     echo "$(date +%FT%T) ping" >> "$log"
25     sleep 10
26 done
```

Listing 6.5: myinit.sh

Die eigentliche Lastverteilung wird durch die haproxy.cfg-Konfiguration gesteuert, die beim Start des Containers in das Verzeichnis /etc/haproxy/ kopiert wird. Diese Datei definiert zwei zentrale Komponenten:

Frontend: Hier wird der öffentliche Port (:80) gebunden, über den die Anwendung erreichbar ist. Eine ACL-Regel (`acl url_direct path_beg /direct`) leitet Anfragen mit dem Pfad /direct an ein separates Backend weiter, das direkt eine Antwort zurückgibt – nützlich für Schnelltests der HAProxy-Infrastruktur.

```
1
2 frontend http-in
3     bind :80
4     acl url_direct path_beg /direct
5     use_backend direct-backend if url_direct
6     default_backend app-backend
7
```

Listing 6.6: Frontend

Backend: Die eigentliche Lastverteilung erfolgt über das app-backend, das die beiden Apache-Container (`apache1:80`, `apache2:80` und `apache3:80`) einbindet. Mittels cookie SERVERID wird eine Sticky Session implementiert, die sicherstellt, dass Nutzer während ihrer Session immer denselben Apache-Container nutzen. Die Strategie roundrobin verteilt neue Sessions gleichmäßig auf alle Server.

```
1         backend direct-backend
2     http-request return status 200 content-type "text/plain" string "Moin from
↪     haproxy\n"
```



```
3
4 backend app-backend
5     balance roundrobin
6     cookie SERVERID insert indirect nocache
7     server apache1 apache1:80 check cookie apache1
8     server apache2 apache2:80 check cookie apache2
9     server apache3 apache3:80 check cookie apache3
```

Listing 6.7: Backend

Des Weiteren wurde in der `start.sh` der Container so konfiguriert, dass er über den **Port 8000** auf dem Host erreichbar ist und gleichzeitig auf den **Port 80** im Container gemappt wird. Diese Port-Weiterleitung ermöglicht es, dass der HAProxy-Container als zentraler Load-Balancer fungiert und Anfragen an die beiden Apache-Container weiterleitet. Dadurch können alle Apache-Container über den Port 8000 erreicht werden, während HAProxy die Lastverteilung und das Routing übernimmt. Dadurch wird sichergestellt, dass die Last gleichmäßig auf die Backend-Server verteilt wird und die Anwendung auch bei hoher Auslastung stabil bleibt. Auch hier beschränkt der Schalter `-cpus=1` die CPU-Leistung des Containers auf 100%.

```
1  #!/bin/bash
2
3  name=haproxy
4  port=8000
5  echo running "$name" "$port"
6  docker container create \
7      --name "$name" \
8      --network mynet \
9      --init \
10     --cpus=1 \
11     -p $port:80 \
12     image-haproxy
13  docker container cp context/myinit.sh $name:/usr/bin
14  docker container cp context/haproxy.cfg $name:/etc/haproxy/
15  docker container start $name
```

Listing 6.8: start.sh

Um die Funktionsweise zu validieren, wurden gezielte Tests durchgeführt:

Beim Aufruf von `http://localhost:8000` antwortet HAProxy direkt mit der Webanwendung, welche von den beiden Apache-Containern bezogen wird und reguliert wird. Bei Lasttests mit `wrk` wurde sichergestellt, dass die Anfragen gleichmäßig auf beide Apache-Container verteilt

werden. Gleichzeitig blieben Sessions dank der Cookie-basierten Sticky Sessions stabil, selbst wenn einer der Container manuell gestoppt wurde. Über den integrierten Stats-Socket (stats socket ipv4@127.0.0.1:9999) konnten wir in Echtzeit verfolgen, wie HAProxy die Last verteilt und ob beide Apache-Container als „healthy“ gemeldet werden. Durch diese Kombination aus Docker-Image, Init-Skript und fein abgestimmter Konfiguration ist HAProxy nicht nur der Türsteher der Anwendung, sondern auch ein zentraler Stabilisator – unsichtbar für die Nutzer, aber unverzichtbar für die Skalierbarkeit und Ausfallsicherheit.

6.3 MariaDB (Datenbank)

Die MariaDB-Datenbank dient als zentrale Speicherlösung für die ToDo-Listen-Daten und die Benutzerverwaltung. Sie gewährleistet eine strukturierte und persistente Speicherung der Aufgaben und Nutzerdaten, sodass sie auch nach einem Neustart des Systems erhalten bleiben. Die Datenbank wurde innerhalb eines Docker-Containers betrieben, wobei das Datenverzeichnis über ein Docker-Volume gesichert wurde, um Datenverlust beim Neustart oder Austausch des Containers zu vermeiden.

6.3.1 Datenbank-Erstellung und Benutzerverwaltung

Beim Aufbau der Umgebung wird zunächst sichergestellt, dass die Datenbank `todo_app` existiert. Falls sie nicht vorhanden ist, wird sie mit folgendem Befehl erstellt:

```
1 CREATE DATABASE IF NOT EXISTS todo_app;
```

Listing 6.9: Erstellen der Datenbank

Damit extern auf die Datenbank zugegriffen werden kann, wurde ein Datenbankbenutzer mit entsprechenden Berechtigungen angelegt:

```
1 CREATE USER IF NOT EXISTS 'dbuser'@'%' IDENTIFIED BY '12345';  
2 GRANT ALL PRIVILEGES ON todoa_pp.* TO 'dbuser'@'%';  
3 FLUSH PRIVILEGES;
```

Listing 6.10: Erstellen des Users

Hierbei wird der Benutzer `dbuser` mit dem Passwort `12345` erstellt, der von überall (`%` steht für jede IP-Adresse) auf die Datenbank zugreifen kann. Durch `GRANT ALL PRIVILEGES` erhält dieser Benutzer vollständige Rechte auf die Datenbank `todo_app`, wodurch er neue Tabellen anlegen, Daten einfügen oder bestehende Einträge bearbeiten kann.

6.3.2 Tabellenstruktur für ToDo-Einträge

Innerhalb der `todo_app`-Datenbank existiert die zentrale Tabelle `todos`, in der die Aufgaben gespeichert werden.

```
1 CREATE TABLE IF NOT EXISTS todos (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   task VARCHAR(255) NOT NULL,  
4   details TEXT NOT NULL,  
5   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
6 );
```

Listing 6.11: Erstellen der Tabelle `todos`

`id INT AUTO_INCREMENT PRIMARY KEY`: Jede Aufgabe erhält eine eindeutige ID, die automatisch hochgezählt wird.

`task VARCHAR(255) NOT NULL`: Die Überschrift der Task, die maximal 255 Zeichen lang sein kann.

`details TEXT NOT NULL`: Die Beschreibung der Task, in der zusätzliche Details gespeichert werden können.

`created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`: Speichert das Erstellungsdatum der Aufgabe, das automatisch mit der aktuellen Uhrzeit belegt wird.

6.3.3 Benutzerverwaltung und Authentifizierung

Zusätzlich zur `todos`-Tabelle gibt es die Tabelle `users`, in der die Benutzerinformationen für das Login hinterlegt sind. Die Tabelle wird mit folgendem SQL-Befehl erstellt:

```
1 CREATE TABLE users (  
2   name VARCHAR(50) PRIMARY KEY,  
3   password VARCHAR(50) NOT NULL  
4 );
```

Listing 6.12: Erstellen der Tabelle `users`

`name VARCHAR(50) PRIMARY KEY`: Der Benutzername dient als eindeutige ID.

password VARCHAR(50) NOT NULL: Das Passwort wird bei uns im Klartext gespeichert, was nicht sicher ist, aber für dieses Projekt ausreicht. Eine sichere Alternative wäre das Hashen des Passworts.

Beim Start der Datenbank wird ein Standardbenutzer mit dem Namen admin und dem Passwort admin angelegt, um den Zugriff auf die Webanwendung zu ermöglichen:

```
1 INSERT INTO users (name, password) VALUES ('admin', 'admin');
```

Listing 6.13: Festlegen der Login-Daten

Dieser Benutzer kann sich in der Webanwendung anmelden und Aufgaben verwalten.

6.4 Redis

Die Redis-Integration in der ToDo-Listen-Anwendung wird zur Verwaltung von Nutzersitzungen eingesetzt. Anstatt Sessions in MariaDB zu speichern, wurde Redis als schnelle und effiziente In-Memory-Datenbank verwendet, um Login-Sitzungen zu verwalten. Dies verbessert die Performance und Skalierbarkeit der Anwendung erheblich, da Redis Zugriffe wesentlich schneller verarbeitet als jedes Mal die Datensätze aus der Datenbank abzurufen um zu überprüfen ob die aktuelle Sitzung noch läuft. Beim Login eines Nutzers wird zunächst überprüft, ob die eingegebenen Zugangsdaten in der MariaDB-Datenbank vorhanden sind. Falls der Benutzername und das Passwort korrekt sind, wird eine eindeutige Session-ID generiert, die anschließend mit einer definierten Ablaufzeit in Redis gespeichert wird. Zur Verwaltung von Benutzersitzungen wird Redis verwendet. Dabei wird eine eindeutige Session-ID generiert und in Redis gespeichert. Der folgende Befehl zeigt, wie die Session mit einer TTL (Time-To-Live) von 3600 Sekunden gespeichert wird:

```
1 SESSION_ID=(uuidgen)redisresult=(uuidgen)redisresult=(redis-cli -h
  → "REDISHOST"-p"REDISHOST"-p"REDIS_PORT" -a "REDISPASSWORD"
  → SETEX"session:REDISPASSWORD" SETEX"session:SESSION_ID" 3600 "$USERNAME")
```

Listing 6.14: Session erstellen

Durch die Verwendung von SETEX wird sichergestellt, dass die Session nach Ablauf der definierten Zeit automatisch gelöscht wird. Dies verhindert, dass Speicherplatz durch inaktive Sitzungen verbraucht wird.

Nach erfolgreichem Speichern der Session in Redis wird ein HTTP-Cookie gesetzt, das die Session-ID enthält. Dieses Cookie wird an den Client übermittelt, sodass der Nutzer für weitere Anfragen authentifiziert bleibt:

```
1 echo "Set-Cookie: session_id=${SESSION_ID}; Path=/; HttpOnly; SameSite=Lax"
2 echo "Status: 303 See Other"
3 echo "Location: /cgi-bin/vns/todo/table3.sh"
```

Listing 6.15: Erfolgreiche Anmeldung

Das Attribut `HttpOnly` stellt sicher, dass das Cookie nur von der Serveranwendung genutzt werden kann und nicht von JavaScript im Browser ausgelesen wird. Dies minimiert Sicherheitsrisiken wie Cross-Site Scripting (XSS). Das Attribut `SameSite=Lax` verhindert, dass das Cookie ungewollt bei externen Anfragen gesendet wird.

Falls Redis das Speichern der Session nicht erfolgreich abschließt, wird eine Fehlermeldung ausgegeben:

```
1 echo "Content-type: text/html"
2 echo ""
3 echo "<html><body><h1>Fehler beim Speichern der Session in Redis</h1></body></html>"
4 exit 1
```

Listing 6.16: Fehlermeldung

Dies kann durch Verbindungsprobleme mit Redis oder fehlerhafte Konfigurationen verursacht werden. In einem solchen Fall kann der Nutzer die Anmeldung erneut versuchen.

Wird ein fehlerhaftes Passwort oder ein nicht vorhandener Benutzername eingegeben, schlägt der Login-Vorgang fehl, und der Nutzer erhält eine Meldung mit einem Link zur erneuten Anmeldung:

```
1 echo "<html>
2 <head><title>Login Failed</title></head> <body> <h1>Login fehlgeschlagen</h1> <a
  ↪ href='/index.html'>Erneut versuchen</a> </body> </html>"
```

Listing 6.17: Login-Fehler

Danach wird in `table3.sh`, also in der ToDo-Listen-Anwendung, die Session-Überprüfung nach der Weiterleitung durchgeführt, um sicherzustellen, dass nur angemeldete Nutzer Zugriff auf die Seite haben. Diese Überprüfung erfolgt durch das Prüfen des HTTP-Cookies und der gespeicherten Session in Redis.

Falls ein Cookie existiert, wird die `session_id` aus dem `HTTP_COOKIE`-Header extrahiert:

```

1 session_id=""
2 if [ -n "HTTP_COOKIE"]; then
3     sessionid=$(echo "$HTTP_COOKIE" | sed -n
4         's/.*session_id=\([^;]*\).*$/\1/p')
5 fi

```

Listing 6.18: Cookie-Extraktion

Falls keine Session-ID gefunden wird, bedeutet dies, dass der Nutzer nicht eingeloggt ist oder seine Sitzung abgelaufen ist. In diesem Fall wird er mit einem HTTP-Redirect zur Login-Seite weitergeleitet:

```

1 if [ -z "$session_id" ]; then
2     echo "Status: 303 See Other"
3     echo "Location: /index.html"
4     echo "Content-type: text/html"
5     echo ""
6     exit 0
7 fi

```

Listing 6.19: Abgelaufene Sitzung

Sollte eine Session-ID vorhanden sein, wird nun in Redis überprüft, ob diese gültig ist. Dies geschieht mit dem folgenden Befehl:

```

1 user=(redis-cli-h"(redis-cli-h"REDIS_HOST" -p
↪ "REDISPORT"-a"REDISPORT"-a"REDIS_PASSWORD" GET "session:$session_id")

```

Listing 6.20: Session-Prüfung

Falls keine gültige Antwort zurückgegeben wird, ist die Session entweder ungültig oder abgelaufen. In diesem Fall erfolgt erneut eine Weiterleitung zur Login-Seite, da der Nutzer nicht authentifiziert ist:

```

1 if [ -z "$user" ]; then
2     echo "Status: 303 See Other"
3     echo "Location: /index.html"
4     echo "Content-type: text/html"
5     echo ""

```

```

6 exit 0
7 fi

```

Listing 6.21: Fehlermeldung

Falls die Session gültig ist und der Benutzer erfolgreich aus Redis abgerufen wurde, wird der Zugriff auf die geschützte ToDo-Seite gewährt. Dadurch bleibt der Nutzer eingeloggt, solange die Session in Redis existiert.

Durch dieses Verfahren wird sichergestellt, dass nur angemeldete Nutzer Zugriff auf die Anwendung haben. Gleichzeitig bleibt die Sitzung speicherschonend, da Redis die Sessions im Arbeitsspeicher hält und automatisch verwaltet. Sollte der Nutzer inaktiv sein, wird die Session nach Ablauf der definierten Zeit automatisch aus Redis entfernt.

6.4.1 Logout-Prozess

In dem Skript `logout.sh` wird zunächst die Session-ID des Benutzers aus dem HTTP-Cookie extrahiert. Dies wird durch den Zugriff auf den Header des HTTP-Requests erreicht, in dem das Cookie gespeichert ist:

```

1 SESSION_ID=(echo"(echo"HTTP_COOKIE" | grep -o 'session_id=[^;]*' | cut -d'=' -f2)

```

Listing 6.22: session-ID extraktion

Falls eine gültige Session-ID im Cookie vorhanden ist, wird diese Session aus Redis gelöscht. Dies wird durch den Redis-Befehl `DEL` erreicht. Die Session-ID wird dabei hinter `session:` eingesetzt, der Redis-Befehl wird ausgeführt und die Ausgabe des Befehls wird verworfen mit `>/dev/null`:

```

1 if [ -n "SESSIONID"];thenredis-cli-h"SESSIONID";thenredis-cli-h"REDIS_HOST" -p
  ↪ "REDISPORT"-a"REDISPORT"-a"REDIS_PASSWORD" DEL "session:$SESSION_ID" >/dev/null
2 fi

```

Listing 6.23: Löschen der ID

Nach der Löschung der Session wird das Cookie im Browser des Nutzers entfernt. Dies geschieht durch das Setzen des Cookies `session_id` auf einen leeren Wert und das Festlegen eines Ablaufdatums in der Vergangenheit (1. Januar 1970). Dies sorgt dafür, dass das Cookie sofort aus dem Browser des Nutzers gelöscht wird:

```
1 echo "Content-type: text/html"
2 echo "Set-Cookie: session_id=; expires=Thu, 01 Jan 1970 00:00:00 GMT; Path=/"
```

Listing 6.24: Session-ID wird auf null gesetzt

Abschließend wird der Benutzer mit einem HTTP-Statuscode 303 (See Other) zur Login-Seite (`index.html`) weitergeleitet:

```
1 echo "Status: 303 See Other"
2 echo "Location: /index.html"
3 echo ""
```

Listing 6.25: Weiterleitung zum Login

Das Skript sorgt also dafür, dass die Session des Nutzers aus Redis gelöscht wird, das Session-Cookie im Browser entfernt wird und der Nutzer auf die Login-Seite weitergeleitet wird. Dies stellt sicher, dass der Benutzer ordnungsgemäß abgemeldet wird.

7 Implementierung und Ergebnisse

In modernen vernetzten Systemen ist die Leistungsfähigkeit und Stabilität der Infrastruktur von entscheidender Bedeutung. Mit der zunehmenden Komplexität von Anwendungen und der steigenden Anzahl gleichzeitiger Nutzer wird es immer wichtiger, die Grenzen der Systeme zu verstehen und sicherzustellen, dass sie auch unter hoher Last zuverlässig funktionieren. Lasttests sind ein unverzichtbares Werkzeug, um die Performance, Skalierbarkeit und Stabilität von Systemen zu bewerten. Sie simulieren reale Belastungsszenarien und helfen, Engpässe, Schwachstellen und Optimierungspotenziale zu identifizieren.

In unserem Projekt haben wir umfangreiche Lasttests durchgeführt, um die Leistungsfähigkeit unserer Infrastruktur zu überprüfen. Dabei haben wir verschiedene Komponenten wie HAProxy, MariaDB und Redis unter die Lupe genommen. Mithilfe von Tools wie wrk, sysbench und k6 haben wir gezielt hohe Last erzeugt und die Reaktion des Systems analysiert. Die Ergebnisse dieser Tests liefern wertvolle Einblicke in das Verhalten unserer Anwendung unter realistischen Bedingungen und zeigen, wie gut das System mit einer großen Anzahl gleichzeitiger Anfragen umgehen kann.

Ein besonderer Fokus lag dabei auf der Datenbankperformance. MariaDB wurde mit intensiven Lese- und Schreiboperationen belastet, um zu überprüfen, wie effizient die Datenbank Anfragen verarbeitet und ob sie unter hoher Last stabil bleibt. Darüber hinaus haben wir die Lastverteilung durch HAProxy getestet, um sicherzustellen, dass Anfragen korrekt an die Backend-Server

weitergeleitet werden und keine Engpässe entstehen. Auch die Performance von Redis als In-Memory-Cache wurde untersucht, um die Geschwindigkeit des Systems bei häufig abgerufenen Daten zu optimieren.

Die Ergebnisse unserer Lasttests zeigen, dass unser System insgesamt stabil und leistungsfähig ist. Allerdings gab es vereinzelt Latenzspitzen und Fehler, die auf Optimierungsbedarf hinweisen. Diese Erkenntnisse sind entscheidend, um die Infrastruktur weiter zu verbessern und sicherzustellen, dass sie auch in Zukunft zuverlässig arbeitet. In den folgenden Abschnitten werden wir die Methodik, Durchführung und Ergebnisse der Lasttests detailliert darstellen und diskutieren.

7.1 MariaDB Stresstest

Um die Leistungsfähigkeit unserer MariaDB-Datenbank unter realistischen Bedingungen zu testen, haben wir einen umfangreichen Stresstest durchgeführt. Ziel war es, herauszufinden, wie gut die Datenbank mit einer hohen Anzahl gleichzeitiger Anfragen zurechtkommt. Dafür haben wir das Tool Sysbench verwendet, das sich hervorragend eignet, um die Performance von Datenbanken unter Last zu messen.

Zunächst haben wir sichergestellt, dass Sysbench auf unserem Work-Container installiert ist. Anschließend haben wir 15 Tabellen mit jeweils 20.000 Datensätzen erstellt, um eine realistische Auslastung zu simulieren. Mit diesem Setup haben wir zwei Haupttests durchgeführt: einen Lese-Last-Test und einen Schreib-Last-Test.

Beim Lese-Last-Test haben wir untersucht, wie schnell die Datenbank auf eine große Anzahl gleichzeitiger SELECT-Abfragen reagiert. Die Ergebnisse waren beeindruckend: Im Durchschnitt wurden 18.502 Abfragen pro Sekunde verarbeitet, was einer Rate von 1.156 Transaktionen pro Sekunde entspricht. Die durchschnittliche Latenz, also die Zeit, die eine Anfrage benötigt, lag bei 17,28 Millisekunden. Besonders erfreulich war, dass 95% der Abfragen in weniger als 34,33 Millisekunden abgeschlossen wurden. Allerdings gab es vereinzelte Spitzenwerte, bei denen die Latenz auf bis zu 217,96 Millisekunden anstieg. Dennoch blieb die Datenbank stabil, und es gab keine Verbindungsabbrüche oder fehlerhaften Abfragen.

Der Schreib-Last-Test konzentrierte sich darauf, wie effizient die Datenbank INSERT- und UPDATE-Abfragen verarbeiten kann. Hier erreichten wir eine Rate von 13.631 Abfragen pro Sekunde bei 2.271 Transaktionen pro Sekunde. Die durchschnittliche Latenz lag bei 8,79 Millisekunden, und 95% der Abfragen wurden in weniger als 17,95 Millisekunden abgeschlossen. Die maximale Latenz betrug hier 199,20 Millisekunden. Auch in diesem Test gab es nur eine einzige Fehlermeldung, die jedoch ignoriert werden konnte, da sie keinen Einfluss auf die Gesamtperformance hatte.

Was bedeuten diese Ergebnisse für uns? Insgesamt zeigt der Test, dass MariaDB auch unter hoher Last stabil und zuverlässig arbeitet. Die hohe Anzahl an Transaktionen pro Sekunde ist ein

klarer Indikator dafür, dass die Datenbank effizient arbeitet. Allerdings haben wir festgestellt, dass es bei extrem hoher Last zu Latenzspitzen kommt, insbesondere beim Lese-Test. Diese Spitzen könnten darauf hinweisen, dass noch Optimierungspotenzial besteht.

Mögliche Ansatzpunkte für Verbesserungen sind die Optimierung von Indizes, um Daten schneller zu finden, die Implementierung von Query-Caching, um wiederholte Anfragen effizienter zu bearbeiten, und eine bessere Lastverteilung, um Engpässe zu vermeiden.

Zusammenfassend lässt sich sagen, dass MariaDB eine solide und leistungsfähige Datenbank ist, die auch unter hoher Belastung stabil läuft. Mit einigen gezielten Optimierungen könnten wir jedoch die Performance weiter verbessern und die Latenzspitzen reduzieren. Dieser Test hat uns wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir die Datenbank noch effizienter machen können.

7.2 Redis Benchmark

Um die Leistungsfähigkeit unseres Redis-Servers zu testen, haben wir einen umfangreichen Benchmark durchgeführt. Ziel war es, herauszufinden, wie gut Redis mit einer hohen Anzahl gleichzeitiger Anfragen umgehen kann. Dafür haben wir das Redis-Benchmark-Tool verwendet, das speziell für solche Lasttests entwickelt wurde.

Der Test bestand aus zwei Hauptkomponenten: SET-Operationen, bei denen Daten in Redis gespeichert werden, und GET-Operationen, bei denen Daten aus Redis abgerufen werden. Um eine realistische Last zu simulieren, haben wir 100.000 Anfragen mit 50 parallelen Clients durchgeführt.

Die Ergebnisse im Detail

Bei den SET-Operationen, also dem Schreiben von Daten in Redis, wurden alle 100.000 Anfragen in nur 1,18 Sekunden abgeschlossen. Das entspricht einem beeindruckenden Durchsatz von 84.388 Anfragen pro Sekunde. Die durchschnittliche Latenz lag bei 0,320 Millisekunden, und 95% aller Anfragen wurden in weniger als 0,471 Millisekunden abgeschlossen. Die maximale Latenz betrug 3,399 Millisekunden. Diese Ergebnisse zeigen, dass Redis Daten extrem schnell speichern kann, mit einer durchschnittlichen Verzögerung von weniger als einer halben Millisekunde.

Bei den GET-Operationen, also dem Abrufen von Daten aus Redis, wurden die 100.000 Anfragen in 1,45 Sekunden abgeschlossen. Der Durchsatz lag hier bei 69.204 Anfragen pro Sekunde, und die durchschnittliche Latenz betrug 0,377 Millisekunden. 95% der Anfragen wurden in weniger als 0,599 Millisekunden abgeschlossen, während die maximale Latenz bei 11,087 Millisekunden lag. Auch hier zeigt sich, dass Redis unglaublich schnell ist, allerdings mit etwas höheren Latenzen als beim Schreiben.

Was bedeuten diese Ergebnisse für uns?

Die Ergebnisse des Benchmarks bestätigen, dass Redis eine extrem leistungsfähige In-Memory-Datenbank ist, die in der Lage ist, eine sehr hohe Anzahl von Anfragen in kürzester Zeit zu verarbeiten. Selbst bei 50 gleichzeitigen Clients bleiben die Antwortzeiten sehr niedrig, was Redis zu einer hervorragenden Wahl für Anwendungen macht, die schnellen Datenzugriff erfordern.

Allerdings haben wir festgestellt, dass die Latenz im GET-Test etwas höher ist als im SET-Test. Dies könnte verschiedene Ursachen haben. Zum einen könnten Cache-Misses eine Rolle spielen, bei denen einige Anfragen nicht direkt im Speicher gefunden werden und daher länger dauern. Zum anderen könnten Netzwerkverzögerungen oder eine höhere Systembelastung durch andere parallel laufende Prozesse zu den leicht erhöhten Latenzen beigetragen haben.

Insgesamt zeigt der Benchmark, dass Redis eine äußerst leistungsfähige und zuverlässige Lösung für Anwendungen ist, die schnellen Datenzugriff erfordern. Die Ergebnisse sind zufriedenstellend und selbst unter hoher Last bleibt Redis stabil und schnell. Mit einigen gezielten Optimierungen, wie der Reduzierung von Cache-Misses oder der Minimierung von Netzwerkverzögerungen, könnten wir die Performance jedoch noch weiter verbessern.

7.3 HAProxy Session-Stickiness

Um sicherzustellen, dass unsere Lastverteilung korrekt funktioniert und Benutzeranfragen während einer aktiven Session immer an denselben Server weitergeleitet werden, haben wir einen umfangreichen Session-Stickiness-Test mit HAProxy durchgeführt. Session Stickiness ist ein entscheidender Faktor, um eine konsistente Benutzererfahrung zu gewährleisten, da sie verhindert, dass Anfragen während einer Session zwischen verschiedenen Backend-Servern hin- und herspringen.

Der Test bestand aus mehreren Schritten, um die Funktionsweise von HAProxy unter die Lupe zu nehmen. Zunächst haben wir überprüft, ob Anfragen desselben Clients tatsächlich immer an denselben Backend-Server weitergeleitet werden. Dazu haben wir zehn aufeinanderfolgende Anfragen gesendet und analysiert, ob sie alle am selben Server ankamen. Zusätzlich haben wir die manuelle Cookie-Setzung getestet, indem wir explizit einen bestimmten Server (apache1) als Ziel vorgaben, um zu sehen, ob HAProxy diese Anweisung korrekt umsetzt.

Der Test begann mit einer ersten Anfrage, bei der das HAProxy-Session-Cookie gespeichert wurde. Dieses Cookie enthält die Information, welcher Backend-Server für die aktuelle Session zuständig ist. Anschließend haben wir zehn weitere Anfragen gesendet, um zu überprüfen, ob HAProxy diese Anfragen tatsächlich an denselben Server weiterleitet. In einem zweiten Schritt haben wir die manuelle Cookie-Setzung getestet, indem wir den Server apache1 explizit als Ziel angaben. Dies diente dazu, die Flexibilität und Genauigkeit des HAProxy-Routings zu überprüfen.

Die Ergebnisse des Tests waren durchweg positiv. HAProxy leitete wiederholte Anfragen konsistent an denselben Server weiter, was bestätigt, dass die Session Stickiness korrekt funktioniert. Die Serverantworten zeigten, dass Apache als Backend-Server verwendet wurde, was die korrekte Funktionsweise der Stickiness unterstrich. Auch die manuelle Cookie-Setzung über `SSERVERID=apache1` funktionierte einwandfrei, und HAProxy leitete die Anfragen wie erwartet an den angegebenen Server weiter. Darüber hinaus blieben die Antwortzeiten stabil und zeigten eine konsistente Geschwindigkeit, was darauf hindeutet, dass unser Load-Balancer effizient arbeitet und keine Engpässe verursacht.

Was bedeuten diese Ergebnisse für uns? Der Test hat gezeigt, dass unser HAProxy-Setup mit Session Stickiness wie erwartet funktioniert. Dies bedeutet, dass Benutzeranfragen während einer Session nicht zwischen den Backend-Servern hin- und hergeschoben werden, was eine konsistente Benutzererfahrung gewährleistet. Gleichzeitig funktioniert die Cookie-basierte Steuerung von HAProxy zuverlässig, sodass wir Anfragen gezielt an bestimmte Server leiten können. Die stabilen Antwortzeiten bestätigen, dass unser Load-Balancer effizient arbeitet und auch unter Last keine Performance-Einbußen verursacht.

Zusammenfassend lässt sich sagen, dass unser HAProxy-Setup mit Session Stickiness korrekt konfiguriert ist und zuverlässig arbeitet. Dies ist ein wichtiger Schritt, um eine konsistente und effiziente Benutzererfahrung zu gewährleisten. Mit diesem Setup können wir sicher sein, dass unsere Lastverteilung auch unter hoher Belastung stabil bleibt und unsere Anwendungen reibungslos funktionieren.

7.4 HAProxy Load-Balancing Test

Um die Leistungsfähigkeit und Stabilität unseres HAProxy-Setups zu überprüfen, haben wir einen umfangreichen Load-Balancing-Test durchgeführt. Für den Test haben wir das Tool `wrk` verwendet, das speziell für HTTP-Lasttests entwickelt wurde. Die Testbedingungen waren so gewählt, dass sie eine realistische Belastung simulieren: 4 Threads, 50 gleichzeitige Verbindungen und eine Testdauer von 10 Sekunden. Die getestete Instanz war unter der URL `http://haproxy:80` erreichbar.

Während des Tests wurden insgesamt 114.414 Anfragen erfolgreich verarbeitet. Die durchschnittliche Latenz lag bei 4,66 Millisekunden, was ein hervorragender Wert ist und die Effizienz unseres Systems unterstreicht. Die maximale Latenz betrug 62,49 Millisekunden, was ebenfalls noch im akzeptablen Bereich liegt. Pro Sekunde wurden im Schnitt 11.418,83 Anfragen bearbeitet, und die Übertragungsrate erreichte 21,03 MB/s. Es gab während des Tests 14 Lese-Fehler, was zwar nicht ideal ist, aber angesichts der Gesamtanzahl an Anfragen nur einen sehr geringen Anteil ausmacht. Verbindungsfehler, Schreibfehler oder Timeout-Probleme traten nicht auf.

Diese Ergebnisse zeigen, dass unser HAProxy-Setup stabil läuft und mit einer hohen Anzahl von Anfragen gut zurechtkommt. Die Performance ist insgesamt sehr stark und verlässlich, was bedeutet, dass wir mit unserer aktuellen Konfiguration auf einem soliden Stand sind. Dennoch

sollten wir die Lese-Fehler im Auge behalten und gegebenenfalls Maßnahmen ergreifen, um diese weiter zu minimieren. Mögliche Ansätze hierfür könnten Optimierungen in der Netzwerkverbindung oder eine Feinjustierung der HAProxy-Konfiguration sein. Zusätzlich könnte es sinnvoll sein, in zukünftigen Tests die Anzahl der Threads und gleichzeitigen Verbindungen zu erhöhen, um zu sehen, wie unser System bei noch stärkerer Belastung reagiert.

7.4.1 Erweiterter Load-Balancing Test unter erhöhter Last

Um unser System noch stärker zu fordern, haben wir einen zweiten Load-Balancing-Test durchgeführt, bei dem wir die Testbedingungen deutlich verschärft haben. Diesmal haben wir die Anzahl der Threads auf 10 und die gleichzeitigen Verbindungen auf 150 erhöht. Der Test lief über einen Zeitraum von 30 Sekunden und wurde ebenfalls mit wrk durchgeführt.

In dieser Zeit wurden insgesamt 317.129 Anfragen verarbeitet und dabei 584,01 MB an Daten übertragen. Die durchschnittliche Latenz betrug 15,24 Millisekunden, wobei die höchste gemessene Latenz bei 249,78 Millisekunden lag. Die Anzahl der Anfragen pro Sekunde belief sich auf 10.922,29, was zeigt, dass unser System auch unter dieser deutlich höheren Last noch eine beeindruckende Performance liefern kann. Allerdings traten während des Tests 5.618 Lese-Fehler und 150 Timeouts auf. Obwohl dies im Vergleich zur Gesamtanzahl an Anfragen nur einen kleinen Prozentsatz ausmacht, deutet es darauf hin, dass das System unter dieser erhöhten Last an seine Grenzen stößt.

Für uns bedeutet das, dass unser HAProxy-Setup auch bei höherer Last grundsätzlich stabil läuft und eine gute Performance bietet. Die durchschnittliche Latenz von 15,24 ms ist zwar etwas höher als beim vorherigen Test, liegt aber noch im akzeptablen Bereich. Die gestiegene Anzahl an Lese-Fehlern und Timeouts zeigt jedoch, dass hier noch Optimierungspotenzial besteht. Mögliche Maßnahmen zur Verbesserung könnten eine weitere Feinjustierung der HAProxy-Konfiguration sein, z. B. durch das Anpassen von Timeouts oder die Erhöhung der maximalen gleichzeitigen Verbindungen. Auch ein genaueres Monitoring des Backend-Systems könnte helfen, Engpässe oder Flaschenhälse zu identifizieren.

7.5 Umfassender Netzwerk- und Performance-Test

Um die Interaktion verschiedener Komponenten unserer Infrastruktur zu analysieren, haben wir einen umfangreichen Netzwerk- und Performance-Test mit unserem Skript tcpdump-alles.sh durchgeführt. Dieses Skript kombiniert mehrere Tools wie TCPDump, wrk, sysbench und redis-benchmark, um die Aktivität und Stabilität unserer Infrastruktur unter realistischen Bedingungen zu prüfen.

Im ersten Schritt wurde die Netzwerkanalyse mit TCPDump gestartet, die sämtliche HTTP-, HTTPS-, MariaDB- und Redis-Anfragen erfasste. Parallel dazu wurde ein HTTP-Lasttest über

HAProxy durchgeführt, bei dem wir mit curl und wrk mehrere GET-Anfragen an HAProxy gesendet haben, um den HTTP-Datenfluss zu simulieren. Danach folgte der MariaDB-Lasttest, bei dem mit sysbench eine intensive Schreib- und Lese-Belastung in der Datenbank erzeugt wurde. Anschließend wurde ein Redis-Benchmark durchgeführt, um die Performance von Redis beim Speichern und Abrufen von Daten zu testen. Abschließend erfolgten gezielte Tests mit Einzelzugriffen auf Redis, bei denen ein Key-Wert-Paar (`todos_cache`) gesetzt und ausgelesen wurde.

Während der gesamten Testphase wurden 1.283.518 Netzwerkpakete erfasst, von denen keine vom Kernel verworfen wurden, was auf eine saubere Paketverarbeitung hinweist. Bei der Analyse der erfassten Daten konnten wir feststellen, dass insgesamt 201.013 HTTP-Anfragen an Port 80 gerichtet wurden, was die hohe Anzahl der GET- und POST-Operationen während der HAProxy-Tests widerspiegelt. Bei den MariaDB-Operationen (Port 3306) wurden 580.952 Anfragen gezählt, was zeigt, dass die Datenbank während der Tests stark belastet wurde. Zusätzlich wurden 400.836 Redis-Operationen (Port 6379) registriert, was auf einen intensiven Cache-Einsatz hinweist.

Die HTTP-Statuscodes zeigten dabei überwiegend erfolgreiche Anfragen: 100.506 Anfragen erhielten den Statuscode 200 (OK), was auf eine fehlerfreie Verarbeitung hinweist. Allerdings gab es auch eine einzelne Anfrage mit dem Statuscode 400 (Bad Request), was auf einen fehlerhaften Request hinweisen könnte. Bei der detaillierten Fehleranalyse wurden 2 Fehler in den Logs erkannt, was in Anbetracht der hohen Gesamtanzahl an Anfragen minimal ist. Zusätzlich wurde 1 Timeout festgestellt, was darauf hindeutet, dass es möglicherweise kurzzeitig zu Verzögerungen in der Antwortzeit gekommen ist.

Insgesamt zeigt der Test, dass unsere Infrastruktur stabil und leistungsfähig ist. Die hohe Anzahl an erfolgreich verarbeiteten Anfragen und der geringe Anteil an Fehlern sprechen für eine gut konfigurierte Umgebung. Die wenigen Fehler und das eine Timeout deuten darauf hin, dass unter sehr hoher Last gelegentlich kleine Verzögerungen auftreten können. Hier könnten Optimierungen an den Timeouts in HAProxy, MariaDB oder Redis vorgenommen werden, um das System noch robuster zu machen. Auch ein genaueres Monitoring der Netzwerkverbindungen könnte sinnvoll sein, um mögliche Flaschenhälse zu erkennen und zu beheben. Insgesamt sind wir mit den Ergebnissen sehr zufrieden, da unser System die umfangreiche Belastung gut bewältigt hat und zuverlässig auf alle Anfragen reagierte. Die Tests haben uns wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir unsere Infrastruktur weiter optimieren können, um zukünftige Lastspitzen noch besser zu bewältigen.

7.6 TCP Dump – Netzwerk- und Performance-Test

Um die Aktivität und Stabilität unserer Infrastruktur unter realistischen Bedingungen zu analysieren, haben wir einen umfangreichen Netzwerk- und Performance-Test mit unserem Skript `tcpdump-alles.sh` durchgeführt. Dieses Skript kombiniert mehrere Tools wie TCPDump, wrk,

sysbench und redis-benchmark, um die Interaktion verschiedener Komponenten wie HAProxy, MariaDB und Redis zu prüfen.

Der Test begann mit einer Netzwerkanalyse mittels TCPDump, die sämtliche HTTP-, HTTPS-, MariaDB- und Redis-Anfragen erfasste. Parallel dazu wurde ein HTTP-Lasttest über HAProxy durchgeführt, bei dem wir mit curl und wrk mehrere GET-Anfragen an HAProxy gesendet haben, um den HTTP-Datenfluss zu simulieren. Anschließend folgte ein MariaDB-Lasttest, bei dem mit sysbench eine intensive Schreib- und Lese-Belastung in der Datenbank erzeugt wurde. Danach wurde ein Redis-Benchmark durchgeführt, um die Performance von Redis beim Speichern und Abrufen von Daten zu testen. Abschließend erfolgten gezielte Tests mit Einzelzugriffen auf Redis, bei denen ein Key-Wert-Paar (*todos_cache*) *gesetzt und ausgelesen wurde*.

Während der gesamten Testphase wurden 1.283.518 Netzwerkpakete erfasst, von denen keine vom Kernel verworfen wurden. Dies deutet auf eine saubere Paketverarbeitung und stabile Netzwerkbedingungen hin. Bei der Analyse der erfassten Daten zeigte sich, dass insgesamt 201.013 HTTP-Anfragen an Port 80 gerichtet wurden, was die hohe Anzahl der GET- und POST-Operationen während der HAProxy-Tests widerspiegelt. Bei den MariaDB-Operationen (Port 3306) wurden 580.952 Anfragen gezählt, was zeigt, dass die Datenbank während der Tests stark belastet wurde. Zusätzlich wurden 400.836 Redis-Operationen (Port 6379) registriert, was auf einen intensiven Cache-Einsatz hinweist.

Die HTTP-Statuscodes zeigten überwiegend erfolgreiche Anfragen: 100.506 Anfragen erhielten den Statuscode 200 (OK), was auf eine fehlerfreie Verarbeitung hinweist. Allerdings gab es auch eine einzelne Anfrage mit dem Statuscode 400 (Bad Request), was möglicherweise auf einen fehlerhaften Request hindeutet. Bei der detaillierten Fehleranalyse wurden 2 Fehler in den Logs erkannt, was in Anbetracht der hohen Gesamtanzahl an Anfragen minimal ist. Zusätzlich wurde 1 Timeout festgestellt, was darauf hindeutet, dass es möglicherweise kurzzeitig zu Verzögerungen in der Antwortzeit gekommen ist.

7.6.1 Erweiterter Netzwerk- und Performance-Test

Um noch detailliertere Einblicke in den Datenverkehr und mögliche Fehlerquellen zu erhalten, haben wir einen erweiterten Netzwerk- und Performance-Test mit unserem Skript `tcpdump-analyse.sh` durchgeführt. Dieses Skript ermöglicht es, die Laufzeit der Analyse flexibel festzulegen. Für diesen Test haben wir eine Laufzeit von 250 Sekunden gewählt.

In dieser Zeit wurden insgesamt 355.313 Pakete erfasst. Alle Pakete wurden erfolgreich verarbeitet, und es gab keine vom Kernel verworfenen Pakete, was auf eine saubere Erfassung und stabile Netzwerkbedingungen hinweist. Bei der detaillierten Analyse der erfassten Daten zeigte sich, dass alle Pakete HTTP-Anfragen über Port 80 betrafen. Insgesamt wurden 355.313 HTTP-Anfragen registriert. Auffällig war, dass es während der gesamten Laufzeit keine MariaDB- oder Redis-Operationen gab. Dies deutet darauf hin, dass diese Dienste während der Analysezeit

entweder nicht aktiv genutzt wurden oder dass der entsprechende Traffic außerhalb der erfassten Ports stattfand.

Die HTTP-Statuscode-Analyse zeigte, dass von den 118.366 erfolgreich abgeschlossenen HTTP-Anfragen fast alle den Statuscode 200 (OK) erhielten. Lediglich eine einzige Anfrage erhielt den Statuscode 400 (Bad Request), was möglicherweise auf einen fehlerhaften Request oder eine falsche URL hindeutet. Bei der Fehleranalyse in den Logs wurden insgesamt 2 Fehler und 1 Timeout festgestellt. Obwohl diese Zahlen im Vergleich zur Gesamtzahl der Anfragen minimal erscheinen, deuten sie darauf hin, dass gelegentliche Verzögerungen oder kleinere Probleme auftreten können. Diese könnten auf temporäre Netzwerkprobleme, einen kurzen Engpass bei der Serverlast oder einen fehlerhaften Client-Request zurückzuführen sein.

7.7 k6 Lasttest – Performance-Analyse von HAProxy und Apache

Um die Performance und Stabilität unserer HAProxy- und Apache-Serverinstanzen zu testen, haben wir einen umfassenden Lasttest mit k6 durchgeführt. Das Hauptziel war es, die Antwortzeiten und die Erfolgsquote der Anfragen zu analysieren sowie potenzielle Engpässe zu identifizieren.

Insgesamt wurden im Testlauf 889.788 HTTP-Anfragen durchgeführt, die sich auf die Dienste HAProxy, Apache1 und Apache2 verteilten. Alle drei Systeme arbeiteten zuverlässig, da 99% der Anfragen erfolgreich bearbeitet wurden. Die Erfolgsquote war damit nahezu fehlerfrei.

Die Analyse der Antwortzeiten zeigte, dass die Mehrheit der Anfragen innerhalb der akzeptablen Grenze von 500 ms blieb. Allerdings gab es einige langsame Anfragen: Bei HAProxy wurden 72 Anfragen, bei Apache1 92 Anfragen und bei Apache2 96 Anfragen registriert, die die Zielzeit von 500 ms überschritten. Dies entspricht jedoch weniger als 1% der Gesamtanfragen, was auf ein insgesamt stabiles und performantes System hinweist.

Die durchschnittliche Antwortzeit betrug 10,43 ms, wobei die langsamsten Anfragen bis zu 2,99 Sekunden benötigten. Die 90% der schnellsten Anfragen lagen unter 21,87 ms, während 95% der Anfragen unter 28,68 ms abgeschlossen wurden. Dies zeigt, dass die überwiegende Mehrheit der Anfragen schnell verarbeitet wurde und nur vereinzelt längere Wartezeiten auftraten.

Insgesamt wurden während des Tests 1,7 GB Daten empfangen und 68 MB Daten gesendet, was einem durchschnittlichen Datendurchsatz von 11 MB/s beim Empfang und 437 kB/s beim Senden entspricht. Die hohe Anzahl an erfolgreichen Anfragen und der stabile Datendurchsatz sprechen dafür, dass unser System auch bei hoher Last effizient arbeitet.

Die durchgeführten Tests zeigen, dass unsere Infrastruktur insgesamt stabil und leistungsfähig ist. Die hohe Anzahl an erfolgreich verarbeiteten Anfragen und der geringe Anteil an Fehlern sprechen für eine gut konfigurierte Umgebung. Die wenigen Fehler und Timeouts deuten darauf hin, dass unter sehr hoher Last gelegentlich kleine Verzögerungen auftreten können. Hier

könnten Optimierungen an den Timeouts in HAProxy, MariaDB oder Redis vorgenommen werden, um das System noch robuster zu machen.

Zusätzlich könnte es sinnvoll sein, in zukünftigen Tests den Traffic von MariaDB und Redis gezielt zu simulieren, um deren Verhalten unter Last genauer zu untersuchen. Auch eine längere Testdauer könnte helfen, sporadisch auftretende Fehler besser zu identifizieren.

Insgesamt sind wir mit den Ergebnissen sehr zufrieden, da unser System die umfangreiche Belastung gut bewältigt hat und zuverlässig auf alle Anfragen reagierte. Die Tests haben uns wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir unsere Infrastruktur weiter optimieren können, um zukünftige Lastspitzen noch besser zu bewältigen.

7.7.1 k6 Login-Lasttest

Um die Stabilität und Performance des Login-Prozesses zu überprüfen, haben wir einen umfangreichen Lasttest mit k6 durchgeführt. Ziel war es, sicherzustellen, dass Benutzer sich erfolgreich anmelden können und dass das Session-Cookie korrekt gesetzt wird. Der Test lief über einen Zeitraum von 2 Minuten mit maximal 100 parallelen virtuellen Nutzern (VUs).

Während des Tests wurden insgesamt 7.506 HTTP-Anfragen gestellt, wobei 100% der Anfragen erfolgreich waren. Es gab keine technischen Fehler oder Abbrüche, was zeigt, dass der Login-Prozess stabil und zuverlässig funktioniert. Bei allen 3.753 Login-Versuchen wurde der Login erfolgreich abgeschlossen, und das Session-Cookie wurde korrekt gesetzt. Dies bestätigt, dass die Authentifizierung sauber umgesetzt ist und einwandfrei funktioniert. Zudem konnte nach dem Login die ToDo-Seite ohne Probleme geladen werden, was die korrekte Funktionsweise der Session-Verwaltung unterstreicht.

Die detaillierte Analyse der Performance-Daten zeigt, dass insgesamt 29 MB an Daten empfangen und 1,6 MB an Daten gesendet wurden. Der durchschnittliche Datendurchsatz lag bei 216 kB/s beim Empfang und 12 kB/s beim Senden. Die durchschnittliche Antwortzeit betrug 443,65 ms, was für eine größere Anzahl von gleichzeitigen Nutzern noch akzeptabel ist. Allerdings gab es einige längere Wartezeiten, wobei die maximale Antwortzeit bei 4,47 Sekunden lag. Die Statistik zeigt, dass 90% der Anfragen schneller als 1,02 Sekunden und 95% der Anfragen schneller als 1,24 Sekunden bearbeitet wurden. Der Test zeigt, dass der Login-Prozess zuverlässig funktioniert und es keine technischen Fehler oder abgebrochenen Anfragen gab. Dies ist ein wichtiger Erfolg, da die Kernfunktionen des Systems stabil und fehlerfrei arbeiten. Allerdings deuten die längeren Antwortzeiten darauf hin, dass unter hoher Last gelegentlich Verzögerungen auftreten können. Hier besteht noch Optimierungspotenzial, um die Performance weiter zu verbessern.

Mögliche Maßnahmen zur Verbesserung könnten die Optimierung der Datenbankabfragen sein, wodurch die Antwortzeiten reduziert werden könnten. Durch effizientere Abfragen ließe sich die Belastung auf die Datenbank verringern, was zu schnelleren Antwortzeiten führen

würde. Eine weitere Möglichkeit ist das Caching von Inhalten. Häufig abgerufene Daten könnten zwischengespeichert werden, um die Last auf dem Server zu verringern und die Zugriffszeiten zu verkürzen. Zusätzlich könnte die Skalierung der Serverressourcen helfen, Engpässe zu vermeiden. Eine bessere Lastverteilung oder zusätzliche Ressourcen könnten die Leistung des Systems unter hoher Belastung verbessern.

Insgesamt zeigt der Test, dass der Login-Prozess stabil arbeitet und alle Authentifizierungen korrekt ablaufen. Die grundlegenden Funktionen, wie das Laden der ToDo-Seite nach dem Login, funktionieren zuverlässig. Trotz der leicht erhöhten Wartezeiten in einigen Fällen können wir den aktuellen Stand als erfolgreich betrachten. Um die Leistung weiter zu verbessern, sollten wir gezielte Optimierungen an den Antwortzeiten und der Skalierbarkeit des Systems vornehmen.

7.7.2 k6 Add-ToDo-Liste

Um die Stabilität und Zuverlässigkeit unserer ToDo-Listen-Funktion unter Belastung zu testen, haben wir einen umfassenden Lasttest mit dem Tool k6 durchgeführt. Ziel war es, herauszufinden, wie die Anwendung bei steigender Nutzerzahl reagiert und ob es zu Verzögerungen oder Fehlern kommt. Der Test lief insgesamt 1 Minute und 27 Sekunden und war in drei Abschnitte unterteilt: Zuerst simulierten wir 20 gleichzeitige Nutzer für 20 Sekunden, um eine moderate Last zu simulieren. Danach erhöhten wir die Anzahl der Nutzer auf 50 für 40 Sekunden, um das System unter höherer Belastung zu testen. Schließlich reduzierten wir die Anzahl der virtuellen Nutzer wieder auf 0, um zu prüfen, wie das System mit abnehmender Last umgeht.

Während des Tests wurden insgesamt 2.007 vollständige Abläufe durchgeführt, bei denen 6.021 HTTP-Anfragen an das System gesendet wurden. Besonders positiv ist, dass 100% der Aufgaben erfolgreich hinzugefügt wurden und die Anzeige der ToDo-Liste einwandfrei funktioniert hat. Dies zeigt, dass unsere Anwendung stabil und zuverlässig arbeitet.

In Bezug auf die übertragenen Daten wurden insgesamt 4,5 MB empfangen und 1,1 MB gesendet. Dies entspricht einem Datendurchsatz von ca. 51 kB/s beim Empfang und 12 kB/s beim Senden.

Die gemessenen Antwortzeiten verdeutlichen, dass unser System auch unter Belastung effizient arbeitet. Die durchschnittliche Antwortzeit betrug lediglich 17,57 ms, was für eine Webanwendung sehr schnell ist. Die kürzeste Antwortzeit lag bei beeindruckenden 357 µs (Mikrosekunden), und selbst die langsamsten Anfragen wurden noch innerhalb von 159 ms verarbeitet, was ebenfalls ein guter Wert ist. Die Statistik zeigt außerdem, dass 90% der Anfragen schneller als 48,3 ms und 95% der Anfragen schneller als 55,04 ms abgeschlossen wurden.

Besonders erfreulich ist, dass es während des gesamten Tests keine Fehler gab. Alle Anfragen wurden erfolgreich bearbeitet, und es traten keine technischen Probleme auf. Dies spricht dafür, dass unser System robust und zuverlässig arbeitet und auch bei hoher Belastung keine Ausfälle

zeigt. Die Skalierung der Nutzerzahlen verlief ebenfalls reibungslos, was zeigt, dass unser System flexibel auf unterschiedliche Lasten reagieren kann.

Was bedeutet das für uns?

Die Tests zeigen, dass die ToDo-Listen-Funktion derzeit sehr stabil und leistungsfähig ist. Die Anwendung verarbeitet Anfragen zuverlässig und schnell, selbst wenn viele Nutzer gleichzeitig aktiv sind. Dass 100 % der Aufgaben erfolgreich hinzugefügt wurden und es keine Fehler gab, zeigt, dass das System technisch sauber arbeitet.

Trotz der sehr positiven Ergebnisse gibt es kleinere Optimierungsmöglichkeiten. Die maximale Antwortzeit von 159 ms könnte durch weitere Verbesserungen reduziert werden. Mögliche Maßnahmen hierfür sind die Optimierung der Datenbankabfragen, um effizientere Abfragen zu ermöglichen, das gezielte Caching häufig abgerufener Daten, um die Last auf dem Server zu verringern, und eine verbesserte Netzwerk-Konfiguration, um Engpässe zu vermeiden.

Fazit

Insgesamt zeigt der Test, dass unsere ToDo-Listen-Funktion stabil und zuverlässig arbeitet. Die Anwendung bewältigt auch unter hoher Last eine große Anzahl von Anfragen ohne Fehler. Mit einigen gezielten Optimierungen könnten wir die Performance weiter verbessern und die Antwortzeiten noch weiter reduzieren. Die Ergebnisse bestätigen, dass unser System technisch gut aufgestellt ist und eine solide Grundlage für zukünftige Erweiterungen bietet.

7.8 Diagramm – Gnuplot: CPU- und RAM-Nutzungsanalyse

Im Rahmen unseres Performance-Monitoring-Tests haben wir ein Skript (`monitor.sh`) verwendet, das kontinuierlich CPU- und RAM-Nutzungsdaten von mehreren Docker-Containern (Apache1, Apache2, Work, HAProxy, Redis und MariaDB) erfasst hat. Die Daten wurden in Intervallen von 1 Sekunde über einen längeren Zeitraum gesammelt und in einer CSV-Datei gespeichert, um sie anschließend zu analysieren und zu visualisieren.

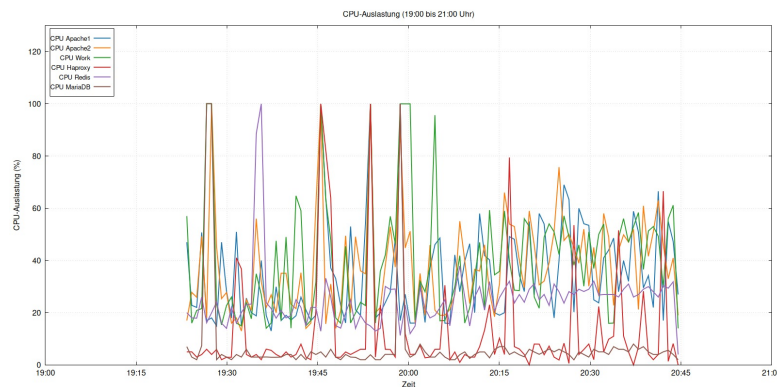


Abbildung 7.1: Gnuplot

Testaufbau

Das Skript `monitor.sh` nutzte den Befehl `docker stats`, um kontinuierlich Daten zu sammeln. Die erfassten Daten umfassten die CPU-Auslastung (%) und den RAM-Verbrauch (MiB) für die Container Apache1, Apache2, Work, HAProxy, Redis und MariaDB. Die Daten wurden in Intervallen von 1 Sekunde erfasst, um detaillierte Schwankungen und Lastspitzen zu erkennen. Die gesammelten Daten wurden in einer CSV-Datei gespeichert und anschließend mithilfe von Gnuplot visualisiert. Das Diagramm zeigt die CPU-Auslastung der verschiedenen Docker-Container zwischen 19:00 Uhr und 21:00 Uhr und gibt uns einen guten Überblick darüber, wie das System unter Belastung reagiert hat.

Allgemeines CPU-Verhalten

Die meisten Container liefen stabil, mit gelegentlichen Spitzen, die bei hoher Last zu erwarten sind. Die CPU-Nutzung blieb meist unter 40%, was auf eine effiziente Ressourcennutzung hindeutet. Allerdings gab es einige Ausreißer, insbesondere bei den Containern Work, HAProxy und MariaDB.

Auffällige Spitzen in der CPU-Nutzung

Die Container Work (grüne Linie) und HAProxy (rote Linie) zeigten teilweise Spitzen von über 100%. Dies deutet darauf hin, dass diese Container bei starker Belastung, z. B. durch viele gleichzeitige Anfragen oder aufwendige Berechnungen, stark beansprucht wurden. Die Spitzen bei Work korrelieren mit unserem `wrk`-Test, der gezielt solche Stresssituationen simulierte. HAProxy hatte ebenfalls hohe Lastspitzen, was darauf hindeutet, dass das Load-Balancing zeitweise stark gefordert war.

Stabile Container

Die Container Apache1 und Apache2 zeigten ein entspanntes Verhalten mit nur gelegentlichen, kleineren Ausreißern. Dies zeigt, dass die Webserver stabil liefen und ihre Aufgaben gut verteilt waren. Redis blieb durchgehend ruhig, was darauf hindeutet, dass der In-Memory-Cache

effizient arbeitete. MariaDB hatte meist niedrige Werte, zeigte aber gelegentlich Spitzen bis zu 60%. Diese Spitzen könnten auf größere Datenbankabfragen oder komplexe Operationen zurückzuführen sein.

Schwankungen und Beruhigung

Gegen Ende des Tests normalisierten sich die CPU-Werte wieder. Dies zeigt, dass sich das System nach der hohen Belastung gut erholte und wieder stabil lief.

Was läuft gut und was könnte besser werden?

Das System zeigte insgesamt ein stabiles Verhalten. Die CPU-Auslastung blieb meist im grünen Bereich, was auf eine effiziente Ressourcennutzung hindeutet. Die CPU-Spitzen bei Work und HAProxy traten vor allem während gezielter Belastungstests auf, was die Belastbarkeit des Systems unterstreicht.

Dennoch gibt es Optimierungspotenzial. Die starken CPU-Spitzen, insbesondere bei Work, könnten durch effizienteren Code, besseres Thread-Management oder eine gezieltere Verteilung der Ressourcen verringert werden. Bei HAProxy könnten feinere Einstellungen beim Load-Balancing und klügere Timeout-Regeln die Belastung auf den Container reduzieren. Für MariaDB könnten Maßnahmen wie bessere Indizes, Query-Caching oder parallele Abfragen helfen, Lastspitzen zu vermeiden.

Fazit

Die Analyse der CPU- und RAM-Nutzung zeigt, dass unser System insgesamt stabil und effizient arbeitet. Die meisten Container bewältigten die Belastung gut, und die CPU-Spitzen traten vor allem während gezielter Stresssituationen auf. Dennoch gibt es Optimierungspotenzial, insbesondere bei der Reduzierung von Lastspitzen und der Feinjustierung der Ressourcenverteilung. Mit gezielten Anpassungen können wir die Performance weiter verbessern und das System noch robuster machen.

8 Fazit und Ausblick

Abbildungsverzeichnis

4.1	OSI-Modell	9
4.2	OSI-Modell und TCP/IP Modell	10
5.1	Die Anwendung	12
5.2	OSI-Modell und TCP/IP Modell	13
5.3	Verzeichnisstruktur (Teil 1)	15
5.4	Verzeichnisstruktur (Teil 2)	16
5.5	Verzeichnisstruktur (Teil 3)	17
7.1	Gnuplot	44
I.1	Poster	50
II.1	Frame mit Listing	53

Tabellenverzeichnis

Listingverzeichnis

6.1	start.sh	19
6.2	Dockerfile	21
6.3	myinit.sh	22
6.4	Dockerfile	23
6.5	myinit.sh	24
6.6	Frontend	24
6.7	Backend	25
6.8	start.sh	25
6.9	Erstellen der Datenbank	26
6.10	Erstellen des Users	26
6.11	Erstellen der Tabelle todos	27
6.12	Erstellen der Tabelle users	27
6.13	Festlegen der Login-Daten	28
6.14	Session erstellen	28
6.15	Erfolgreiche Anmeldung	29
6.16	Fehlermeldung	29
6.17	Login-Fehler	29
6.18	Cookie-Extraktion	30
6.19	Abgelaufene Sitzung	30
6.20	Session-Prüfung	30
6.21	Fehlermeldung	31
6.22	session-ID extraktion	31
6.23	Löschen der ID	31
6.24	Session-ID wird auf null gesetzt	32
6.25	Weiterleitung zum Login	32
II.1	Beamer Frame	51
II.2	Beamer Frame mit Listing	52
III.1	Einbinden von subfiles	54
III.2	Erstellen von subfiles	54

Anhang

I Die Poster

Passend zu diesem Template existieren noch zwei weitere Vorlagen für Poster. Diese unterscheiden sich lediglich durch die Ausrichtung der Seite und sind ansonsten identisch. Der Aufbau der Dateien und Projektstruktur ist weitestgehend identisch zu dieser Vorlage, es gilt lediglich auf einige kleine Besonderheiten zu verweisen.

In der sog. `multicols` Umgebung ist es nicht möglich, mit Floatumgebungen zu arbeiten, dies ist für fast alle Beispiele aus dieser Anleitung irrelevant, lediglich Grafiken müssen anders eingebunden werden. Dazu finden sich jedoch Beispiele in den Vorlagen. Ebenso ist ein Teil der Präambel in eine separate Datei ausgelagert, in dieser sollten auch nicht ohne weiteres Änderungen vorgenommen werden. Wie in diesem Template, müssen innerhalb der Präambel Zitierstil, sowie Autor:innen und Titel gesetzt werden.

Die Vorlagen können auf meiner Webseite sowohl angeschaut, als auch heruntergeladen werden, alle Hinweise zum Kompilieren gelten für die Poster ebenso. Ergebnis dieser Vorlage ist z. B. folgendes Poster:

Eine unnötig komplizierte und sehr sehr lange Abhandlung

Untersuchung einer Sache von wirklichem Wert

Maxi Mustermensch

Einleitung

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Zielstellung

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Befehl	Ergebnis
<code>\tiny</code>	klein
<code>\scriptsize</code>	klein
<code>\footnotesize</code>	etwas größer
<code>\small</code>	noch etwas größer
<code>\normalsize</code>	normal
<code>\large</code>	groß
<code>\LARGE</code>	größer
<code>\Huge</code>	riesig
<code>\huge</code>	noch riesiger

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Materialien und noch etwas, damit es zweizeilig wird

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Hochschule Bremerhaven

sie eine falsche Anmutung vermitteln.

```
for i in {1..10}; do
  echo "81"
done
```

Listing 1: Ein Listing

Schluss

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.



Abbildung 1: Bildunterschrift [1, S.14]

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjft – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Referenzen

- [1] M. M. Mustermensch „MS Windows NT Kernel Description.“ de. (Dez. 2012). Adresse: <http://www.800multimedia.com/wins/ntkernel.html> (besucht am 30.09.2012).

Abbildung I.1: Poster

II Beamer

$\text{\LaTeX}$ bietet nicht nur die Möglichkeit schriftliche Ausarbeitungen in Form von Hausarbeiten zu erstellen, sondern mit Beamer eine Alternative zu PowerPoint oder Keynote. Das Layout ist in vielen Fällen schlicht, was nicht negativ gemeint ist und erinnert im Design eher an die frühen 2000er. Durch einige Anpassungen ist es dennoch möglich ein schlichtes, aber funktionales Layout zu entwerfen, welches sich gut für Präsentationen eignet.

Eine solche Vorlage findet sich analog zu den Poster Vorlagen und erfordert wenig Umgewöhnung. Es können Tabellen und Abbildungen, sowie Referenzen wie gewohnt genutzt werden. Lediglich die Listings erfordern eine kleine Umgewöhnung, ebenso wie das Arbeiten mit der *column* Umgebung.

```

1 \section{Frame Example with table}
2
3 \begin{frame}{Frame Example with table}
4   \begin{columns}
5
6     \begin{column}{0.4\textwidth}
7       This is some text in the second frame~\autocite{donovan2015go}.
8       This is some text in the second~\autocite{kane2018docker}.
9     \end{column}
10
11    \begin{column}{0.59\textwidth}
12      \scriptsize{
13        \begin{talltblr}[caption={\LaTeX~Sonderzeichen}]{colspec={X[1,m] X[0.5,c,m]},
14          ↪ rowhead=1, width=0.8\textwidth}\toprule
15        Befehl                & Ergebnis                & \\\ \midrule
16        \textbackslash\&      & \&                      & \\\ \cmidrule{1-2}
17        \textbackslash\%      & \%                      & \\\ \cmidrule{1-2}
18        \textbackslash\$      & \$                      & \\\ \cmidrule{1-2}
19        \textbackslash\#      & \#                      & \\\ \cmidrule{1-2}
20        \textbackslash\{      & \{                      & \\\ \cmidrule{1-2}
21        \textbackslash\}      & \}                      & \\\ \cmidrule{1-2}
22        \end{talltblr}}
23    \end{column}
24  \end{columns}
25 \end{frame}

```

Listing II.1: Beamer Frame

Wie zu sehen, wird in Beamer ebenfalls mit sections gearbeitet. Um einen neuen Slide zu erstellen wird `\begin{frame}` genutzt. Die `columns` Umgebung ermöglicht das Unterteilen des Frames in mehrere Spalten, welche daraufhin als einzelne `column` definiert werden. Dabei kann die Textbreite angegeben werden.

Listings müssen über eine besondere Umgebung definiert und eingebunden werden, da Beamer ansonsten nicht mit `minted` umgehen kann.

```
1 \defverbatim[colored]\CodeOne{
2   \begin{minted}{python}
3     import numpy as np
4     import pylab as pl
5
6     def f_x(x):
7         return np.exp(x)+x**2-5*x
8
9     def approx_f(x):
10        return 1 -4*x +3./2*x**2
11
12    xvals = np.arange(-4,4,0.1)
13    fx_vals = [f_x(x) for x in xvals]
14    approx_vals = [approx_f(x) for x in xvals]
15
16    pl.plot(xvals,fx_vals)
17    pl.plot(xvals,approx_vals)
18
19    pl.show()
20 \end{minted}
21 \captionof{listing}{Example Code}
22 }
23
24 \section{Code Example}
25 \begin{frame}{Code Example}
26   Some Text on first Frame~\autocite{gregg2013systems}
27   \CodeOne
28   \vspace{0.5cm}
29 \end{frame}
```

Listing II.2: Beamer Frame mit Listing

Wie im Listing gezeigt, wird das Listing außerhalb des Frames definiert und in Zeile 27 eingefügt. CodeOne ist dabei der zugewiesene Name unter dem das Listing eingefügt werden kann. Die Definition erfolgt in Zeile 1, der Name ist frei wählbar. Das Listing so einzubinden ist nicht optimal, jedoch bleibt aktuell keine andere Möglichkeit, wenn mit `minted` gearbeitet wird. Das Inhaltsverzeichnis und die Referenzen werden automatisch erstellt, alle Einstellungen können natürlich innerhalb der Vorlage angepasst werden. Obiges Listing würde mit der Vorlage folgendermaßen aussehen. Wie immer finden sich alle Beispiele auch in den jeweiligen Vorlagen.

Code Example

Some Text on first Frame [1]

```

1  import numpy as np
2  import pylab as pl
3
4  def f_x(x):
5      return np.exp(x)+x**2-5*x
6
7  def approx_f(x):
8      return 1 -4*x +3./2*x**2
9
10 xvals = np.arange(-4,4,0.1)
11 fx_vals = [f_x(x) for x in xvals]
12 approx_vals = [approx_f(x) for x in xvals]
13
14 pl.plot(xvals,fx_vals)
15 pl.plot(xvals,approx_vals)
16
17 pl.show()

```

Listing: Example Code

**Hochschule
Bremerhaven**

Maxi Musterfrau (12345), Maxi Mustermann (54321)
Titel
10. August 2021
4 / 12

Abbildung II.1: Frame mit Listing

III Quickbuild

Wird ein Projekt mit der Zeit relativ groß, verfügt also über eine Vielzahl von Listings, Tabellen, Abbildungen und Text, dann wird für das Kompilieren eine nicht unerhebliche Zeit benötigt. Um das ganze Dokument, inkl. aller Referenzen und Abkürzungen etc. zu bauen, ist dies auch nötig und lässt sich nicht vermeiden. Oft benötigt man jedoch nur einen kurzen Blick auf Layout oder den Umfang des bisher geschriebenen und möchte nicht das Dokument in seiner Gesamtheit kompilieren.

Für diese Anforderungen gibt es sog. subfiles, diese ermöglichen es, nur einen kleinen Teil des Dokumentes zu kompilieren und als PDF zu betrachten. Dabei kommt es zu gewissen Einschränkungen, so werden Seitenzahlen, Referenzen und Abkürzungen nicht korrekt dargestellt, dennoch erhält man einen guten Blick auf das Layout. Um mit subfiles zu arbeiten, muss einiges beachtet werden. So muss statt des `\input{}` das Kommando `\subfile{}` genutzt werden. Darüber hinaus müssen alle `.tex`-Dateien mit einer eigenen `documentclass` ausgestattet werden. In der hauptdatei `.tex` sieht dies dann folgendermaßen aus.

```
1 \subfile{src/einleitung.tex}
2 \subfile{src/struktur.tex}
3 \subfile{src/zurhauptdatei.tex}
```

Listing III.1: Einbinden von subfiles

Jedes Subfile muss dann mit der passenden `documentclass` beginnen.

```
1 \documentclass[../hauptdatei.tex]{subfiles}
2 \begin{document}
3 \section{Beispiel}
4
5 Hier könnte sinnvoller Inhalt stehen.
6
7 \end{document}
```

Listing III.2: Erstellen von subfiles

Durch die eigene `documentclass`, welche wiederum auf die `hauptdatei.tex` verweist und die Angabe von `\begin` bzw. `\end{document}` kann die entsprechende `tex`-Datei separat kompiliert werden. Dafür kann das Skript `quickbuild.sh` genutzt werden, welches als Argument den Namen der zu kompilierenden Datei erwartet und ein PDF erzeugt.

Abbildungen, Listings, Tabellen können wie gewohnt genutzt werden. Wird `buildlualatex.sh` ausgeführt, baut nach wie vor das gesamte Dokument inkl. aller Subfiles und Referenzen.

Selbstständigkeitserklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit habe ich mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegt.

Bremerhaven, den 10. März 2025

Unterschrift: